

## SOLIDWORKS API (.NET VBA C#) — Examples (18/50): Create\_Width\_Mate\_Example\_CSharp- Display\_Configuration\_Description\_in\_Bil

SOLIDWORKS API Documentation (converted from CHM)

## Create and Edit Width Mate Example (C#)

This example shows how to create and edit a width mate.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo1.sldasm
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a width mate of constraint type, dimension.
// 2. Press F5 to continue.
// 3. Changes the width mate to constraint type, percent.
// 4. Inspect the Immediate window and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace WidthMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private AssemblyDoc swAssy;
        private WidthMateFeatureData swWidthMateData;
        private MateFeatureData swMateData;
        private SelectionMgr swSelMgr;
        private bool boolstatus;
        private Feature feat;
        private Face2[] faceTab = new Face2[2];
        private Face2[] facesWidth = new Face2[2];
        private object vFacesTab;
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            swMateData = (MateFeatureData)swAssy.CreateMateData(11); //Width mate

            if (swMateData.TypeName == (int)swMateType_e.swMateWIDTH)
            {
                swWidthMateData = (WidthMateFeatureData)swMateData;
                swWidthMateData.ConstraintType = (int)swMateWidthOptions_e.swMateWidth_Dimension;

                Debug.Print("Width Mate Constraint Type is " + swWidthMateData.ConstraintType);

                // Select faces for the width and tab of this width mate
                boolstatus = swModel.Extension.SelectByRay(0.00868857956595548, 0.0414144214960288, 0.0633435410960033, -0.520148774
                boolstatus = swModel.Extension.SelectByRay(0.040068521476087, 0.0399799509449394, 0.0585753748188154, 0.320596315934
                boolstatus = swModel.Extension.SelectByRay(0.0313565896258297, 0.0296508617577445, 0.0442099188854286, 0.34052466687
                boolstatus = swModel.Extension.SelectByRay(0.0267766714922573, 0.0259424893815421, 0.0380784753310763, -0.8497256923

                facesWidth[0] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
                facesWidth[1] = (Face2)swSelMgr.GetSelectedObject6(2, -1);

                object vFacesWidth = facesWidth;

                swWidthMateData.WidthSelection = vFacesWidth;

                faceTab[0] = (Face2)swSelMgr.GetSelectedObject6(3, -1);
                faceTab[1] = (Face2)swSelMgr.GetSelectedObject6(4, -1);

                vFacesTab = faceTab;

                swWidthMateData.TabSelection = (vFacesTab);

                if (swWidthMateData.ConstraintType == (int)swMateWidthOptions_e.swMateWidth_Dimension)
                {
                    swWidthMateData.DistanceFromEnd = 0.04975511004;
                    swWidthMateData.FlipDimension = true;
                }

                feat = (Feature)swAssy.CreateMate(swWidthMateData);
            }

            swModel.GraphicsRedraw2();

            System.Diagnostics.Debugger.Break();
        }
    }
}
```

```

    feat = swModel.Extension.GetLastFeatureAdded();
    swMateData = (MateFeatureData)feat.GetDefinition();
    if (swMateData.TypeName == (int)swMateType_e.swMateWIDTH)
    {
        swWidthMateData = (WidthMateFeatureData)swMateData;

        swWidthMateData.ConstraintType = (int)swMateWidthOptions_e.swMateWidth_Percent;
        swWidthMateData.PercentDistanceFromEnd = 47;
        swWidthMateData.FlipDimension = false;
        Debug.Print("Width mate constraint type changed to " + swWidthMateData.ConstraintType);
        Debug.Print("Distance from the end of this width mate: " + swWidthMateData.DistanceFromEnd);
        Debug.Print("Percentage of distance from the end of this width mate: " + swWidthMateData.PercentDistanceFromEnd);
        Debug.Print("Move entities to opposite sides of the dimension? " + swWidthMateData.FlipDimension);

        boolstatus = feat.ModifyDefinition(swWidthMateData, swAssy, null);
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

## Create and Edit Width Mate Example (VBA)

This example shows how to create and edit a width mate.

```
'-----
' Preconditions:
' 1. Open:
'   public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo1.sldasm
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a width mate of constraint type, dimension.
' 2. Press F5 to continue.
' 3. Changes the width mate to constraint type, percent.
' 4. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

## Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swWidthMateData As SldWorks.WidthMateFeatureData
Dim swMateData As SldWorks.MateFeatureData
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim feat As SldWorks.Feature
Dim faceTab(1) As SldWorks.Face2
Dim facesWidth(1) As SldWorks.Face2
Dim vFacesTab As Variant

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager

    Set swMateData = swAssy.CreateMateData(11) 'Width mate

    If (swMateData.TypeName = SwConst.swMateType_e.swMateWIDTH) Then

        Set swWidthMateData = swMateData
        swWidthMateData.ConstraintType = SwConst.swMateWidthOptions_e.swMateWidth_Dimension

        Debug.Print "Width Mate Constraint Type is " & swWidthMateData.ConstraintType

        ' Select faces for the width and tab of this width mate
        boolstatus = swModel.Extension.SelectByRay(8.68857956595548E-03, 4.14144214960288E-02, 6.33435410960033E-02,
        -0.520148774728431, -0.59141018013918, -0.616181183562315, 4.68381592786756E-04, 2, True, 1, 0)
        boolstatus = swModel.Extension.SelectByRay(0.040068521476087, 3.99799509449394E-02, 5.85753748188154E-02,
        0.320596315934938, -0.422312467890091, -0.847862124212143, 4.68381592786756E-04, 2, True, 1, 0)
        boolstatus = swModel.Extension.SelectByRay(3.13565896258297E-02, 2.96508617577445E-02, 4.42099188854286E-02,
        0.340524666870961, -0.380278973953885, -0.859901653226112, 4.31713609895031E-04, 2, True, 16, 0)
        boolstatus = swModel.Extension.SelectByRay(2.67766714922573E-02, 2.59424893815421E-02, 3.80784753310763E-02,
        -0.849725692314326, -0.107993323108602, 0.516046209156602, 4.31713609895031E-04, 2, True, 16, 0)

        Set facesWidth(0) = swSelMgr.GetSelectedObject6(1, -1)
        Set facesWidth(1) = swSelMgr.GetSelectedObject6(2, -1)

        Dim vFacesWidth As Variant
        vFacesWidth = facesWidth

        swWidthMateData.WidthSelection = vFacesWidth

        Set faceTab(0) = swSelMgr.GetSelectedObject6(3, -1)
        Set faceTab(1) = swSelMgr.GetSelectedObject6(4, -1)

        vFacesTab = faceTab

        swWidthMateData.TabSelection = (vFacesTab)

        If (swWidthMateData.ConstraintType = SwConst.swMateWidthOptions_e.swMateWidth_Dimension) Then
            swWidthMateData.DistanceFromEnd = 0.04975511004
            swWidthMateData.FlipDimension = True
        End If

        Set feat = swAssy.CreateMate(swWidthMateData)

    End If
```

```

swModel.GraphicsRedraw2

Stop

Set feat = swModel.Extension.GetLastFeatureAdded

Set swMateData = feat.GetDefinition

If (swMateData.TypeName = SwConst.swMateType_e.swMateWIDTH) Then
    Set swWidthMateData = swMateData

    Debug.Print "Width mate constraint type is " & swWidthMateData.ConstraintType

    swWidthMateData.ConstraintType = SwConst.swMateWidthOptions_e.swMateWidth_Percent
    swWidthMateData.PercentDistanceFromEnd = 47
    swWidthMateData.FlipDimension = False
    Debug.Print "Width mate constraint type changed to " & swWidthMateData.ConstraintType
    Debug.Print "Distance from the end of this width mate: " & swWidthMateData.DistanceFromEnd
    Debug.Print "Percentage of distance from the end of this width mate: " & swWidthMateData.PercentDistanceFromEnd
    Debug.Print "Move entities to opposite sides of the dimension? " & swWidthMateData.FlipDimension

    boolstatus = feat.ModifyDefinition(swWidthMateData, swAssy, Nothing)

End If
End Sub

```

## Create Wrap Feature on Multiple Faces Example (C#)

This example shows how to create a wrap feature on multiple faces.

```
//-----
// Preconditions: Verify that the part to open exists.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects the plane on which to sketch a circle.
// 3. Sketches the circle.
// 4. Selects the sketch of the circle and the faces on which to
//    wrap it.
// 5. Creates the wrap feature.
// 6. Examine the FeatureManager design tree and part.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\molds\\telephone.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Select the plane on which to sketch the circle for the wrap feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Plane8", "PLANE", 0, 0, 0, false, 0, null, 0);

            //Sketch the circle
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(-0.035, 0.011624, 0.0, -0.031081, 0.018171, 0.0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            //Select the sketch of the circle and the faces on which to wrap it
            //Because the type of wrap feature to create Scribe, no pull direction entity is selected
            status = swModelDocExt.SelectByID2("Sketch30", "SKETCH", 0, 0, 0, false, 4, null, 0);
            status = swModelDocExt.SelectByRay(-0.103709743982563, 0.00466186411857746, 0.0465727951450701, 1, 0, 0, 0.000421383417784414, 2, true, 1, 0);
            status = swModelDocExt.SelectByRay(-0.105251033879711, 0.0013155840361718, 0.0360382097004597, 1, 0, 0, 0.000421383417784414, 2, true, 1, 0);
            status = swModelDocExt.SelectByRay(-0.104507668954227, 0.00255494702965538, 0.0257514968545461, 1, 0, 0, 0.000421383417784414, 2, true, 1, 0);
            status = swModelDocExt.SelectByRay(-0.101403318635789, 0.0181709207475484, 0.0255036242558494, 1, 0, 0, 0.000421383417784414, 2, true, 1, 0);
            status = swModelDocExt.SelectByRay(-0.100395783628869, 0.0205257104351672, 0.0356664008024147, 1, 0, 0, 0.000421383417784414, 2, true, 1, 0);
            status = swModelDocExt.SelectByRay(-0.0997494761213602, 0.0190384748429869, 0.0484318396352955, 1, 0, 0, 0.000421383417784414, 2, true, 1, 0);

            //Create the wrap feature
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.InsertWrapFeature2((int)swWrapSketchType_e.swWrapSketchType_Scribe, 0.00254, false,
(int)swWrapMethods_e.swWrapMethods_SplineSurface, 5);

            swModel.ClearSelection2(true);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

## Create Wrap Feature on Multiple Faces Example (VBA)

This example shows how to create a wrap feature on multiple faces.

```

'-----
' Preconditions: Verify that the part to open exists.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects the plane on which to sketch a circle.
' 3. Sketches the circle.
' 4. Selects the sketch of the circle and the faces on which to
'    wrap it.
' 5. Creates the wrap feature.
' 6. Examine the FeatureManager design tree and part.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\molds\telephone.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select the plane on which to sketch the circle for the wrap feature
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Plane8", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

    'Sketch the circle
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchManager.CreateCircle(-0.035, 0.011624, 0#, -0.031081, 0.018171, 0#)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    'Select the sketch of the circle and the faces on which to wrap it
    'Because the type of wrap feature to create is Scribe, no pull direction entity is selected
    status = swModelDocExt.SelectByID2("Sketch30", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    status = swModelDocExt.SelectByRay(-0.103709743982563, 4.66186411857746E-03, 4.65727951450701E-02, 1, 0, 0, 4.21383417784414E-04, 2, True, 1, 0)
    status = swModelDocExt.SelectByRay(-0.105251033879711, 1.3155840361718E-03, 3.60382097004597E-02, 1, 0, 0, 4.21383417784414E-04, 2, True, 1, 0)
    status = swModelDocExt.SelectByRay(-0.104507668954227, 2.55494702965538E-03, 2.57514968545461E-02, 1, 0, 0, 4.21383417784414E-04, 2, True, 1, 0)
    status = swModelDocExt.SelectByRay(-0.101403318635789, 1.81709207475484E-02, 2.55036242558494E-02, 1, 0, 0, 4.21383417784414E-04, 2, True, 1, 0)
    status = swModelDocExt.SelectByRay(-0.100395783628869, 2.05257104351672E-02, 3.56664008024147E-02, 1, 0, 0, 4.21383417784414E-04, 2, True, 1, 0)
    status = swModelDocExt.SelectByRay(-9.97494761213602E-02, 1.90384748429869E-02, 4.84318396352955E-02, 1, 0, 0, 4.21383417784414E-04, 2, True, 1, 0)

    'Create the wrap feature
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.InsertWrapFeature2(swWrapSketchType_e.swWrapSketchType_Scribe, 0.00254, False,
    swWrapMethods_e.swWrapMethods_SplineSurface, 5)

    swModel.ClearSelection2 True

End Sub

```

## Create Wrap Feature on Multiple Faces Example (VB.NET)

This example shows how to create a wrap feature on multiple faces.

```

'-----
' Preconditions: Verify that the part to open exists.
'-----
' Postconditions:
' 1. Opens the part.
' 2. Selects the plane on which to sketch a circle.
' 3. Sketches the circle.
' 4. Selects the sketch of the circle and the faces on which to
'    wrap it.
' 5. Creates the wrap feature.
' 6. Examine FeatureManager design tree and part.
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\molds\telephone.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select the plane on which to sketch the circle for the wrap feature
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Plane8", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

        'Sketch the circle
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchManager.CreateCircle(-0.035, 0.011624, 0.0#, -0.031081, 0.018171, 0.0#)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        'Select the sketch of the circle and the faces on which to wrap it
        'Because the type of wrap feature to create is Scribe, no pull direction entity is selected
        status = swModelDocExt.SelectByID2("Sketch30", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        status = swModelDocExt.SelectByRay(-0.103709743982563, 0.00466186411857746, 0.0465727951450701, 1, 0, 0, 0.000421383417784414, 2, True, 1, 0)
        status = swModelDocExt.SelectByRay(-0.105251033879711, 0.0013155840361718, 0.0360382097004597, 1, 0, 0, 0.000421383417784414, 2, True, 1, 0)
        status = swModelDocExt.SelectByRay(-0.104507668954227, 0.00255494702965538, 0.0257514968545461, 1, 0, 0, 0.000421383417784414, 2, True, 1, 0)
        status = swModelDocExt.SelectByRay(-0.101403318635789, 0.0181709207475484, 0.0255036242558494, 1, 0, 0, 0.000421383417784414, 2, True, 1, 0)
        status = swModelDocExt.SelectByRay(-0.100395783628869, 0.0205257104351672, 0.0356664008024147, 1, 0, 0, 0.000421383417784414, 2, True, 1, 0)
        status = swModelDocExt.SelectByRay(-0.0997494761213602, 0.0190384748429869, 0.0484318396352955, 1, 0, 0, 0.000421383417784414, 2, True, 1, 0)

        'Create the wrap feature
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.InsertWrapFeature2(swWrapSketchType_e.swWrapSketchType_Scribe, 0.00254, False, swWrapMethods_e.swWrapMethods_SplineSu
rface, 5)

        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```



## Crop Drawing View Using Jagged Outline Example (C#)

This example shows how to crop a drawing view using a jagged outline with the least shape intensity.

```
//-----
// Preconditions: Verify that the drawing to open exists.
//
// Postconditions:
// 1. Opens the drawing.
// 2. Selects and activates Drawing View1 on Sheet1.
// 3. Creates a rectangle.
// 4. Crops Drawing View1 around the rectangle.
// 5. Examine the drawing.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawing = default(DrawingDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            View swView = default(View);
            SketchManager swSketchMgr = default(SketchManager);
            object[] sketchLines = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddr
w";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOption
s_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swDrawing = (DrawingDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Crop Drawing View1 using a jagged outline
            //with the least shape intensity
            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            status = swDrawing.ActivateView("Drawing View1");
            sketchLines = (object[])swSketchMgr.CreateCornerRectangle(-0.0574722079408937, 0.0331536511827661, 0, 0.0371399698368841,
-0.0373161088172339, 0);
            errors = swView.Crop2(true, false, 5);

            swModel.ClearSelection2(true);

            if (swView.CropViewNoOutline)
            {
                Debug.Print("No outline.");
            }
            else
            {
                if (swView.CropViewJaggedOutline)
                {
                    Debug.Print("Jagged outline with shape intensity (1=most to 5=least): " + swView.CropViewJaggedShapeIntensity);
                }
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

## Crop Drawing View Using Jagged Outline Example (VBA)

This example shows how to crop a drawing view using a jagged outline with the least shape intensity.

```
' -----
' Preconditions: Verify that the drawing to open exists.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Selects and activates Drawing View1 on Sheet1.
' 3. Creates a rectangle.
' 4. Crops Drawing View1 around the rectangle.
' 5. Examine the drawing.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
' -----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim swSelMgr As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim swSketchMgr As SldWorks.SketchManager
Dim sketchLines As Variant
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Option Explicit

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swDrawing = swModel
    Set swSelMgr = swModel.SelectionManager

    'Crop Drawing View1 using a jagged outline
    'with the least shape intensity
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketchMgr = swModel.SketchManager
    status = swDrawing.ActivateView("Drawing View1")
    sketchLines = swSketchMgr.CreateCornerRectangle(-5.74722079408937E-02, 3.31536511827661E-02, 0, 3.71399698368841E-02, -3.73161088172339E-02, 0)
    errors = swView.Crop2(True, False, 5)

    swModel.ClearSelection2 True

    If swView.CropViewNoOutline Then
        Debug.Print "No outline."
    Else
        If swView.CropViewJaggedOutline Then
            Debug.Print "Jagged outline with shape intensity (1=most to 5=least): " & swView.CropViewJaggedShapeIntensity
        End If
    End If

End Sub
```

## Crop Drawing View Using Jagged Outline Example (VB.NET)

This example shows how to crop a drawing view using a jagged outline with the least shape intensity.

```
'-----
' Preconditions: Verify that the drawing to open exists.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Selects and activates Drawing View1 on Sheet1.
' 3. Creates a rectangle.
' 4. Crops Drawing View1 around the rectangle.
' 5. Examine the drawing.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim swSelMgr As SelectionMgr
        Dim swView As View
        Dim swSketchMgr As SketchManager
        Dim sketchLines As Object
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swDrawing = swModel
        swSelMgr = swModel.SelectionManager

        'Crop Drawing View1 using a jagged outline
        'with the least shape intensity
        status = swModelDocExt.SelectByID("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        swView = swSelMgr.GetSelectedObject6(1, -1)
        swSketchMgr = swModel.SketchManager
        status = swDrawing.ActivateView("Drawing View1")
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.0574722079408937, 0.0331536511827661, 0, 0.0371399698368841, -0.0373161088172339, 0)
        errors = swView.Crop2(True, False, 5)

        swModel.ClearSelection2(True)

        If swView.CropViewNoOutline Then
            Debug.Print("No outline.")
        Else
            If swView.CropViewJaggedOutline Then
                Debug.Print("Jagged outline with shape intensity (1=most to 5=least): " & swView.CropViewJaggedShapeIntensity)
            End If
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

## Crop View Example (C#)

This example shows how to crop a view.

```
//-----
// Preconditions: Ensure the specified drawing document exists.
//
// Postconditions: The view is cropped around the specified rectangle.
//
// NOTE: Because the model is used elsewhere, do not save changes when exiting.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CropView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        View myView;
        DrawingDoc drawingDoc;
        bool boolstatus;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.sldrw", 3, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("foodprocessor - Sheet1", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            myView = (View)((SelectionMgr)Part.SelectionManager).GetSelectedObject6(1, -1);

            drawingDoc = (DrawingDoc)Part;
            boolstatus = drawingDoc.ActivateView("Drawing View1");
            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0574722079408937, 0.0331536511827661, 0, 0.0371399698368841,
-0.0373161088172339, 0);

            longstatus = myView.Crop();
            Part.ClearSelection2(true);
        }

        public SldWorks swApp;
    }
}
```

### Crop View Example (VBA)

This example shows how to crop a view.

```
'-----  
' Preconditions: Ensure the specified drawing document exists.  
'  
' Postconditions: The view is cropped around the specified rectangle.  
'  
' NOTE: Because the model is used elsewhere, do not save changes when exiting.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim myView As SldWorks.View  
Dim boolstatus As Boolean  
Dim longstatus As Long, longwarnings As Long  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
  
    Set Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", 3, 0, "", longstatus, longwarnings)  
    swApp.ActivateDoc2 "foodprocessor - Sheet1", False, longstatus  
    Set Part = swApp.ActiveDoc  
  
    boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)  
    Set myView = Part.SelectionManager.GetSelectedObject6(1, -1)  
  
    boolstatus = Part.ActivateView("Drawing View1")  
    Dim vSkLines As Variant  
    vSkLines = Part.SketchManager.CreateCornerRectangle(-5.74722079408937E-02, 3.31536511827661E-02, 0, 3.71399698368841E-02,  
-3.73161088172339E-02, 0)  
  
    longstatus = myView.Crop  
    Part.ClearSelection2 True  
  
End Sub
```

## Crop View Example (VB.NET)

This example shows how to crop a view.

```
'-----
' Preconditions: Ensure the specified drawing document exists.
' Postconditions: The view is cropped around the specified rectangle.
'
' NOTE: Because the model is used elsewhere, do not save changes when exiting.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim myView As View
    Dim boolstatus As Boolean
    Dim longstatus As Long, longwarnings As Long
```

```
    Sub main()
```

```
        Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", 3, 0, "", longstatus, longwarnings)
        swApp.ActivateDoc2("foodprocessor - Sheet1", False, longstatus)
        Part = swApp.ActiveDoc

        boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        myView = Part.SelectionManager.GetSelectedObject6(1, -1)

        boolstatus = Part.ActivateView("Drawing View1")
        Dim vSkLines As Object
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0574722079408937, 0.0331536511827661, 0, 0.0371399698368841,
-0.0373161088172339, 0)

        longstatus = myView.Crop
        Part.ClearSelection2(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

## Cut and Paste Sketch Example (C#)

This example shows how to cut and paste a sketch to and from the Microsoft Windows Clipboard.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a sketch containing three lines.
// 3. Press F5 to continue.
// 4. Cuts the sketch and places it on the Microsoft
// Windows Clipboard.
// 5. Press F5 to continue.
// 6. Pastes the sketch from the Clipboard into the part
// document.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace EditCutCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SketchManager swSketchManager;
        SketchSegment swSketchSegment;
        bool status;

        int errors;

        public void Main()
        {
            // Create a new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prtdot", (int)swDwgPaperSizes_e.swDwgPap
erASize, 0, 0);
            swApp.ActivateDoc3("Part1", false, (int)swRebuildOnActivation_e.swRebuildActiveDoc, ref errors);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Create a sketch
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.066124, 0.011735, 0.0, -0.039675, 0.011735, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.039675, -0.008754, 0.0, -0.010245, -0.008754, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.010245, -0.029989, 0.0, 0.022166, -0.029989, 0.0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            System.Diagnostics.Debugger.Break();
            // Examine the graphics area to
            // verify that the sketch was created

            // Press F5 to continue executing the
            // macro

            // Select the sketch and place it on the Microsoft Windows Clipboard
            status = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.051595524691358, 0.011734722222222, 0, false, 0, null, 0);
            swModel.EditCut();

            System.Diagnostics.Debugger.Break();
            // Examine the graphics area to
            // verify that the sketch was cut

            // Press F5 to continue executing the
            // macro

            // Paste the contents of the Clipboard into the
            // part document

            swModel.Paste();
            // Examine the graphics area to
            // verify that the sketch was pasted
            // into the document
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

## Cut and Paste Sketch Example (VBA)

This example shows how to cut and paste a sketch to and from the Microsoft Windows Clipboard.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a sketch containing three lines.
' 3. Press F5 to continue.
' 4. Cuts the sketch and places it on the Microsoft
'    Windows Clipboard.
' 5. Press F5 to continue.
' 6. Pastes the sketch from the Clipboard into the part
'    document.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim status As Boolean
Dim errors As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Create a new part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", swDwgPaperSizes_e.swDwgPaperASize, 0, 0)
    swApp.ActivateDoc3 "Part1", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors
    Set swModelDocExt = swModel.Extension

    ' Create a sketch
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchManager.CreateLine(-0.066124, 0.011735, 0#, -0.039675, 0.011735, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.039675, -0.008754, 0#, -0.010245, -0.008754, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.010245, -0.029989, 0#, 0.022166, -0.029989, 0#)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    Stop
    ' Examine the graphics area to
    ' verify that the sketch was created

    ' Press F5 to continue executing the
    ' macro

    ' Select the sketch and place it on the Microsoft Windows Clipboard
    status = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.051595524691358, 1.17347222222222E-02, 0, False, 0, Nothing, 0)
    swModel.EditCut

    Stop
    ' Examine the graphics area to
    ' verify that the sketch was cut

    ' Press F5 to continue executing the
    ' macro

    ' Paste the contents of the Clipboard into the
    ' part document
    swModel.Paste

    ' Examine the graphics area to
    ' verify that the sketch was pasted
    ' into the document

End Sub
```



## Cut and Paste Sketch Example (VB.NET)

This example shows how to cut and paste a sketch to and from the Microsoft Windows Clipboard.

```

-----
' Preconditions: Verify that the specified part template exists.
-----
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a sketch containing three lines.
' 3. Press F5 to continue.
' 4. Cuts the sketch and places it on the Microsoft
'    Windows Clipboard.
' 5. Press F5 to continue.
' 6. Pastes the sketch from the Clipboard into the part
'    document.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSketchManager As SketchManager
    Dim swSketchSegment As SketchSegment
    Dim status As Boolean
    Dim errors As Integer

    Sub Main()

        ' Create a new part document
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", swDwgPaperSizes_e.swDwgPaperASize, 0, 0)
        swApp.ActivateDoc3("Part1", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors)
        swModelDocExt = swModel.Extension

        ' Create a sketch
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchManager.CreateLine(-0.066124, 0.011735, 0.0#, -0.039675, 0.011735, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.039675, -0.008754, 0.0#, -0.010245, -0.008754, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.010245, -0.029989, 0.0#, 0.022166, -0.029989, 0.0#)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        Stop
        ' Examine the graphics area to
        ' verify that the sketch was created

        ' Press F5 to continue executing the
        ' macro

        ' Select the sketch and place it on the Microsoft Windows Clipboard
        status = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.051595524691358, 0.0117347222222222, 0, False, 0, Nothing, 0)
        swModel.EditCut()

        Stop
        ' Examine the graphics area to
        ' verify that the sketch was cut

        ' Press F5 to continue executing the
        ' macro

        ' Paste the contents of the Clipboard into the
        ' part document

        swModel.Paste()
        ' Examine the graphics area to
        ' verify that the sketch was pasted
        ' into the document
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

## Cut Body and Keep All Bodies Example (C#)

This example shows how to cut a body and keep all bodies.

```
//-----
// Preconditions:
// 1. Verify that the specified part document template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a body.
// 3. Splits the body into two bodies.
// 4. Examine the graphics area and Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public PartDoc swPart;
        ModelDoc2 Part;
        bool boolstatus;
        Feature Feature;

        public void Main()
        {
            //Open new part document
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\part.prtdot", 0, 0, 0);

            //Set up event
            swPart = (PartDoc)Part;
            AttachEventHandlers();

            //Create body
            CreateBodiesAndSketch();
            boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 0, null, 0);
            Feature = (Feature)Part.FeatureManager.FeatureCut3(true, false, false, (int)swEndConditions_e.swEndCondThroughAll,
            (int)swEndConditions_e.swEndCondBlind, 0.01, 0.01, false, false, false, 0.01745329251994, 0.01745329251994, false, false,
            false, false, false, true, true, false, false, false, (int)swStartConditions_e.swStartSketchPlane, 0, false);
            if ((Feature == null))
            {
                Debug.Print("No feature created.");
            }
        }

        public void CreateBodiesAndSketch()
        {
            //Create body
            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.06869486923422, 0.06291203863612, -0.006492164309718, false, 0, null, 0);
            Part.ClearSelection2(true);
            Part.SketchRectangle(-0.0424567617866, 0.0388405707196, 0, 0.05638579404467, -0.03750124069479, 0, false);
            Part.ShowNamedView2("Trimetric", 8);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            Part.FeatureManager.FeatureExtrusion3(true, false, false, 0, 0, 0.12, 0.01, false, false, false,
            false, 0.01745329251994, 0.01745329251994, false, false, false, false, false, false, false,
            0, 0, false);
            Part.ClearSelection2(true);

            //Create sketch for cut feature
            boolstatus = Part.Extension.SelectByID2("", "FACE", -0.02909828822015, 0.03884057071963, 0.09843602253397, false, 0, null, 0);
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            object[] vSkLines = null;
            vSkLines = (object[])Part.SketchManager.CreateCornerRectangle(-0.0628943705795, -0.07743122635196, 0, 0.1160562766823, -0.04532565168643, 0);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);

        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swPart.PromptBodiesToKeepNotify += this.swPart_PromptBodiesToKeepNotify;
        }

        private int swPart_PromptBodiesToKeepNotify(object swFeat, ref object bodies)
        {
            Debug.Print("PartDoc_PromptBodiesToKeepNotify fired.");
            Feature theFeature = default(Feature);
            object[] bodiesArr = null;
            bodiesArr = (object[])bodies;
            if ((swFeat != null))
            {
                theFeature = (Feature)swFeat;
                object[] bodiesToKeep = new object[1];
                //Change BodyOption to Body1 or Body2 to show other options
                string BodyOption = null;
                BodyOption = "AllBodies";
                switch (BodyOption)
                {
                    case "AllBodies":
                        theFeature.SetBodiesToKeep(true, bodiesToKeep, (int)swInConfigurationOpts_e.swThisConfiguration, null);
                }
            }
        }
    }
}
```

```

        break;
    case "Body1":
        bodiesToKeep[0] = bodiesArr[0];
        theFeature.SetBodiesToKeep(false, bodiesToKeep, (int)swInConfigurationOpts_e.swThisConfiguration, null);
        break;
    case "Body2":
        bodiesToKeep[0] = bodiesArr[1];
        theFeature.SetBodiesToKeep(false, bodiesToKeep, (int)swInConfigurationOpts_e.swThisConfiguration, null);
        break;
    }
    return 1;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

## Cut Body and Keep All Bodies Example (VBA)

This example shows how to cut a body and keep all bodies.

```
'-----
' Preconditions:
' 1. Copy and paste this code in the main module.
' 2. Click Insert > Class module and copy and paste this code in the class module.
' 3. Verify that the specified part document template exists.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a body.
' 3. Splits the body into two bodies.
' 4. Examine the graphics area and Immediate window.
'-----

'Main module

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim Feature As SldWorks.Feature
Dim PartEvents As Class1

Sub main()

    Set swApp = Application.SldWorks

    'Open new part document
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\part.prtdot", 0, 0, 0)

    'Set up event
    Set PartEvents = New Class1
    Set PartEvents.swPartDoc = swApp.ActiveDoc

    'Create body
    Call CreateBodiesAndSketch

    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Set Feature = Part.FeatureManager.FeatureCut3(True, False, False, swEndCondThroughAll, swEndCondBlind, 0.01, 0.01, False, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, False, False, True, True, False, False, False, swStartSketchPlane, 0, False)
    If (Feature Is Nothing) Then
        Debug.Print "No feature created."
    End If

End Sub

Sub CreateBodiesAndSketch()

    'Create body
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.06869486923422, 0.06291203863612, -0.006492164309718, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    Part.SketchRectangle -0.0424567617866, 0.0388405707196, 0, 0.05638579404467, -0.03750124069479, 0, 1
    Part.ShowNamedView2 "Trimetric", 8
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    Part.FeatureManager.FeatureExtrusion3 True, False, False, 0, 0, 0.12, 0.01, False, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, False, False, False, 0, 0, False
    Part.ClearSelection2 True

    'Create sketch for cut feature
    boolstatus = Part.Extension.SelectByID2("", "FACE", -0.02909828822015, 0.03884057071963, 0.09843602253397, False, 0, Nothing, 0)
    Part.SketchManager.InsertSketch True
    Part.ClearSelection2 True
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0628943705795, -0.07743122635196, 0, 0.1160562766823, -0.04532565168643, 0)
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)

End Sub

Back to top

'Class module

Option Explicit

Public WithEvents swPartDoc As SldWorks.PartDoc

Public Function swPartDoc_PromptBodiesToKeepNotify(ByVal swFeat As Object, ByRef bodies As Variant) As Long
    Debug.Print "PartDoc.PromptBodiesToKeepNotify fired."
    Dim theFeature As SldWorks.Feature
    If Not swFeat Is Nothing Then
        Set theFeature = swFeat
        Dim bodiesToKeep(0) As Object
        'Change BodyOption to Body1 or Body2 to show other options
        Dim BodyOption As String
```

```
BodyOption = "AllBodies"
Select Case BodyOption
  Case "AllBodies"
    theFeature.SetBodiesToKeep True, bodiesToKeep, swThisConfiguration, Nothing
  Case "Body1"
    Set bodiesToKeep(0) = bodies(0)
    theFeature.SetBodiesToKeep False, bodiesToKeep, swThisConfiguration, Nothing
  Case "Body2"
    Set bodiesToKeep(0) = bodies(1)
    theFeature.SetBodiesToKeep False, bodiesToKeep, swThisConfiguration, Nothing
End Select
End If

swPartDoc_PromptBodiesToKeepNotify = 1

End Function
```

[Back to top](#)

## Cut Body and Keep All Bodies Example (VB.NET)

This example shows how to cut a body and keep all bodies.

```

-----
Preconditions:
1. Verify that the specified part document template exists.
2. Open the Immediate window.
-----
Postconditions:
1. Opens a new part document.
2. Creates a body.
3. Splits the body into two bodies.
4. Examine the graphics area and Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim boolstatus As Boolean
    Dim Feature As Feature
    Public WithEvents swPart As PartDoc

    Public Sub main()

        'Open new part document
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\part.prtdot", 0, 0, 0)

        'Set up event
        swPart = Part
        AttachEventHandlers()

        'Create body
        Call CreateBodiesAndSketch()
        boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        Feature = Part.FeatureManager.FeatureCut3(True, False, False, swEndConditions_e.swEndCondThroughAll,
swEndConditions_e.swEndCondBlind, 0.01, 0.01, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False,
False, False, True, True, False, False, False, swStartConditions_e.swStartSketchPlane, 0, False)
        If (Feature Is Nothing) Then
            Debug.Print("No feature created.")
        End If

    End Sub

    Sub CreateBodiesAndSketch()
        'Create body
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.06869486923422, 0.06291203863612, -0.006492164309718, False, 0, Nothing, 0)
        Part.ClearSelection2(True)
        Part.SketchRectangle(-0.0424567617866, 0.0388405707196, 0, 0.05638579404467, -0.03750124069479, 0, 1)
        Part.ShowNamedView2("Trimetric", 8)
        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        Part.FeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.12, 0.01, False, False, False, False, 0.01745329251994,
0.01745329251994, False, False, False, False, False, False, 0, 0, False)

        Part.ClearSelection2(True)

        'Create sketch for cut feature
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.02909828822015, 0.03884057071963, 0.09843602253397, False, 0, Nothing, 0)
        Part.SketchManager.InsertSketch(True)
        Part.ClearSelection2(True)
        Dim vSkLines As Object
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0628943705795, -0.07743122635196, 0, 0.1160562766823, -0.04532565168643, 0)
        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swPart.PromptBodiesToKeepNotify, AddressOf Me.swPart_PromptBodiesToKeepNotify
    End Sub

    Private Function swPart_PromptBodiesToKeepNotify(ByVal swFeat As Object, ByRef bodies As Object) As Integer
        Debug.Print("PartDoc_PromptBodiesToKeepNotify fired.")
        Dim theFeature As Feature
        If Not swFeat Is Nothing Then
            theFeature = swFeat
            Dim bodiesToKeep(0) As Object
            'Change BodyOption to Body1 or Body2 to show other options
            Dim BodyOption As String
            BodyOption = "AllBodies"
            Select Case BodyOption
                Case "AllBodies"
                    theFeature.SetBodiesToKeep(True, bodiesToKeep, swInConfigurationOpts_e.swThisConfiguration, Nothing)
                Case "Body1"
                    bodiesToKeep(0) = bodies(0)
                    theFeature.SetBodiesToKeep(False, bodiesToKeep, swInConfigurationOpts_e.swThisConfiguration, Nothing)
                Case "Body2"
                    bodiesToKeep(0) = bodies(1)
                    theFeature.SetBodiesToKeep(False, bodiesToKeep, swInConfigurationOpts_e.swThisConfiguration, Nothing)
            End Select
        End If
    End Function
End Class

```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

## Cut Body in Half using Macro Feature Example (VBA)

This example shows how to cut a body in a part document in half using a macro feature.

```
'-----
' Preconditions:
' 1. Create a VBA macro.
' 2. Click Tools > References > SolidWorks version exposed type libraries for
'    add-in use > OK.
' 3. Copy Macros to the main module and rename it Macros. (To rename the module, click
'    View > Properties Window and type Macros in (Name).)
' 4. Click Insert > Module and copy PropMgrConst to that module and name it PropMgrConst.
' 5. Click Insert > Class module and copy PropMgr to that class module and name it PropMgr.
' 6. Click Insert > Class module and copy PropMgrCmd to that class module and name it PropMgrCmd.
' 7. Click Insert > Class module and copy PropMgrHdlr to that class module and name it PropMgrHdlr.
' 8. Open public_documents\samples\tutorial\api\plate.sldprt.

' Postconditions:
' 1. Click the part in the graphics area, which is then displayed in Surface on Mass Center in the
'    PropertyManager page.
' 2. Click the green check mark in the PropertyManager page to insert the macro feature.
' 3. Cuts the body in half and adds the AngleSurf1 macro feature to the FeatureManager design tree.
' 4. Right-click AngleSurf1 and select Edit Feature on the context-sensitive menu.
' 5. Change 1 to 2 in Select the body to keep in the AngleSurf1 PropertyManager page.
' 6. Click the green check mark in the PropertyManager page.
' 7. Displays the opposite side of the body in the graphics area.
' 8. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

```
'Macros
```

```
Option Explicit
```

```
' Macro feature rebuild function
```

```
Function swmMain(swAppIn, partIn, featureIn)
    Dim featData As SldWorks.MacroFeatureData
    Dim Config As SldWorks.Configuration
    Dim ConfigName As String
```

```
    Set featData = featureIn.GetDefinition
```

```
    ' Get name of configuration being rebuilt
    Set Config = featData.CurrentConfiguration
    ConfigName = Config.Name
```

```
    Dim sels, types, selmarks
    Dim body1 As SldWorks.Body2
    Call featData.GetSelections3(sels, types, selmarks, Nothing, Nothing)
    If IsEmpty(sels) Then
        swmMain = "Body has not been selected!"
        Exit Function
    End If
```

```
    If sels(0) Is Nothing Then
        swmMain = "Body has not been selected!"
        Exit Function
    End If
```

```
    Set body1 = sels(0)
```

```
    Dim modeler As SldWorks.modeler
    Set modeler = swAppIn.GetModeler
    Dim props As Variant
    props = body1.GetMassProperties(1)
    Dim p1(0 To 2) As Double
    Dim v1(0 To 2) As Double
    Dim v2(0 To 2) As Double
```

```
    p1(0) = props(0)
    p1(1) = props(1)
    p1(2) = props(2)
    v1(0) = 0
    v1(1) = 0
    v1(2) = 1
    v2(0) = 1
    v2(1) = 0
    v2(2) = 0
```

```
    Dim surf As SldWorks.Surface
    Set surf = modeler.CreatePlanarSurface2(p1, v1, v2)
    Dim box As Variant
    box = body1.GetBodyBox
```

```
    Dim uvLow, uvHigh As Variant
    uvLow = surf.GetClosestPointOn(box(0), box(1), box(2))
    uvHigh = surf.GetClosestPointOn(box(3), box(4), box(5))
```

```
    Dim midPt(0 To 2) As Double
    Dim i As Integer
    For i = 0 To 2
        midPt(i) = (uvLow(i) + uvHigh(i)) / 2
    Next i
```

```
    For i = 0 To 2
        uvLow(i) = (uvLow(i) - midPt(i)) * 1.1 + midPt(i)
        uvHigh(i) = (uvHigh(i) - midPt(i)) * 1.1 + midPt(i)
    Next i
```



```

uvLow = surf.GetClosestPointOn(uvLow(0), uvLow(1), uvLow(2))
uvHigh = surf.GetClosestPointOn(uvHigh(0), uvHigh(1), uvHigh(2))

Dim sheet As SldWorks.Body2
Dim uv(0 To 3) As Double
uv(0) = uvLow(3)
uv(1) = uvHigh(3)
uv(2) = uvLow(4)
uv(3) = uvHigh(4)
Set sheet = modeler.CreateSheetFromSurface(surf, uv)

' Transform with angle
Dim mathUtil As SldWorks.MathUtility
Set mathUtil = swAppIn.GetMathUtility
Dim aXform As SldWorks.MathTransform
Dim basePt As SldWorks.MathPoint
Dim RetVal As Boolean
Set basePt = mathUtil.CreatePoint(midPt)
Dim xAxis As MathVector
Set xAxis = mathUtil.CreateVector(v2)
Set aXform = mathUtil.CreateTransformRotateAxis(basePt, xAxis, 3.1416159 / 2)
RetVal = sheet.ApplyTransform(aXform)

' Assign edge ID
Dim edges As Variant
Dim faces As Variant
featData.GetEntitiesNeedUserId sheet, faces, edges
edges = sheet.GetEdges
For i = 0 To UBound(edges)
    featData.SetEdgeUserId edges(i), i, 0
    Dim id1 As Long
    Dim id2 As Long
    featData.GetEdgeUserId edges(i), id1, id2
Next i

Dim editBodies As Variant
Dim editBdy As Body2
Dim resBody As Body2
editBodies = featData.editBodies
Set editBdy = editBodies(0)

Dim result As Variant
Dim err As Long
result = editBdy.Operations2(SWBODYCUT, sheet, err)
Dim wb As Long
featData.GetIntegerByName "WhichBody", wb
Set resBody = result(wb - 1)

Set swmMain = resBody
resBody.Hide partIn
End Function

' Macro feature edit definition function
Sub swmPM(swAppIn, partIn, featureIn)
    Dim swPage As New PropMgr
    swPage.Init swAppIn, partIn, featureIn, swCmdEdit, swAppIn.GetCurrentMacroPathName
    swPage.Show
End Sub

'Inserts macro feature with customized PropertyManager page
Public Sub swmInsertCustomizedMacroFeature()
    Dim swAppIn, partIn, featureIn
    Set swAppIn = Application.SldWorks
    Set partIn = swAppIn.ActiveDoc

    If partIn.GetType() <> swDocPART Then
        MsgBox ("Available only from part document!")
        Exit Sub
    End If

    Dim swPage As New PropMgr
    swPage.Init swAppIn, partIn, featureIn, swCmdCreate, swAppIn.GetCurrentMacroPathName
    swPage.Show
End Sub

```

[Back to top](#)

'PropMgrConst

```

Public Enum swPageCmdState
    swCmdCreate = 1
    swCmdEdit = 2
End Enum

Public Const ID_GROUP = 1
Public Const ID_SELECTION = 2
Public Const ID_NUMBER = 3

```

[Back to top](#)

'PropMgr

```

Option Explicit

Private m_swApp As SldWorks.SldWorks
Private m_Part As SldWorks.ModelDoc2
Private m_feature As SldWorks.feature
Private m_Page As PropertyManagerPage2
Private m_Group As PropertyManagerPageGroup
Private m_Selection As PropertyManagerPageSelectionbox
Private m_NumberBox As PropertyManagerPageNumberbox
Private m_Text As PropertyManagerPageTextbox
Private m_swPageCmd As New PropMgrCmd

```

```

Private m_cmdState As swPageCmdState
Private m_pageHdlr As New PropMgrHdlr

Private Sub Layout()
    Dim swControl As PropertyManagerPageControl
    Dim title As String
    Dim buttonTypes As Long
    Dim message As String
    Dim id As Long
    Dim controlType As Integer
    Dim caption As String
    Dim alignment As Integer
    Dim options As Long
    Dim tip As String
    Dim filterArray(0 To 1) As Long

    m_pageHdlr.Init Me

    If m_cmdState = swCmdCreate Then
        title = "Cut body macro feature"
    Else
        title = m_feature.Name
    End If

    buttonTypes = swPropertyManagerOptions_OkayButton + swPropertyManagerOptions_CancelButton + swPropertyManagerOptions_LockedPage
    Dim errorh As Long
    Set m_Page = m_swApp.CreatePropertyManagerPage(title, buttonTypes, m_pageHdlr, errorh)
    If Not m_Page Is Nothing Then

        'Initial set up of the message in the PropertyManager page
        Message = "Select body to cut, then select the side of the body to keep."
        m_Page.SetMessage Message, swImportantMessageBox

        'Begin adding the required controls to the PropertyManager

        'Group box
        Id = ID_GROUP
        caption = "Surface on Mass Center"
        options = swGroupBoxOptions_Visible + swGroupBoxOptions_Expanded + swGroupBoxOptions_Checked
        Set m_Group = m_Page.AddGroupBox(Id, caption, options)
        If Not m_Group Is Nothing Then
            'Selection box
            Id = ID_SELECTION
            controlType = swControlType_Selectionbox
            caption = "Selection box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Select body"
            Set swControl = m_Group.AddControl2(Id, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Selection = swControl
                filterArray(0) = swSelSOLIDBODIES
                filterArray(1) = swSelSURFACEBODIES
                m_Selection.SingleEntityOnly = True
                m_Selection.Height = 50
                m_Selection.SingleEntityOnly = True
                m_Selection.SetSelectionFilters (filterArray)
                m_Selection.SetStandardPictureLabel swBitmapLabel_SelectFaceSurface
            End If

            'Selection box
            Id = ID_SELECTION
            controlType = swControlType_Numberbox
            caption = "Number box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Select the body to keep"
            Set swControl = m_Group.AddControl2(Id, controlType, caption, alignment, options, tip)

            If Not swControl Is Nothing Then
                Set m_NumberBox = swControl
                m_NumberBox.SetRange swNumberBox_UnitlessInteger, 1, 2, 1, True
                m_NumberBox.Value = GetCmd().GetNumberValue()
            End If
        End If
    End If

End Sub

Public Sub Show()
    m_Page.Show
End Sub

Sub Init(swApp, part, feature, cmdState As swPageCmdState, macroPath As String)
    Set m_swApp = swApp
    Set m_Part = part

    If Not IsEmpty(feature) Then
        Set m_feature = feature
    End If

    m_cmdState = cmdState
    m_Part.ClearSelection2 True
    m_swPageCmd.Init swApp, part, feature, cmdState, macroPath
    Layout
End Sub

Public Function GetCmd() As PropMgrCmd
    Set GetCmd = m_swPageCmd
End Function

```

[Back to top](#)

```
'PropMgrCmd

Option Explicit

Private m_swApp As SldWorks.SldWorks
Private m_Part As SldWorks.ModelDoc2
Private m_feature As SldWorks.feature
Private m_bodySel As Object
Private m_whichBody As Long
Private m_featData As SldWorks.MacroFeatureData
Private m_modelComp As SldWorks.Component2
Private m_cmdState As swPageCmdState
Private m_macroPath As String

Public Sub Init(swApp, part, feature, cmdState As swPageCmdState, macroPath As String)
    Set m_swApp = swApp
    Set m_Part = part
    m_macroPath = macroPath
    m_whichBody = 1

    If Not IsEmpty(feature) Then
        Set m_feature = feature
    End If

    If cmdState = swCmdEdit Then ' On Edit Definition

        Dim ret As Boolean
        Set m_featData = m_feature.GetDefinition
        Set m_modelComp = m_feature.GetComponent
        m_cmdState = cmdState
        ret = m_featData.AccessSelections(m_Part, m_modelComp)
        Dim sels, types, selmarks

        Call m_featData.GetSelections3(sels, types, selmarks, Nothing, Nothing)

        If Not IsEmpty(sels) And Not sels(0) Is Nothing Then
            Set m_bodySel = sels(0)
            Call m_bodySel.Select(True, selmarks(0))
        End If

        m_featData.GetIntegerByName "WhichBody", m_whichBody
    End If
End Sub

Public Sub OnOk()
    If m_cmdState = swCmdEdit Then ' On Edit Definition

        Dim sels, types, selmarks

        Call m_featData.GetSelections3(sels, types, selmarks, Nothing, Nothing)

        Dim newSels(0 To 0) As Body2
        Dim newSelMarks(0 To 0) As Long

        Set newSels(0) = m_bodySel
        newSelMarks(0) = 0
        sels = newSels
        selmarks = newSelMarks

        Dim DrViews(0 To 0) As View
        Set DrViews(0) = Nothing

        Call m_featData.SetSelections2(sels, selmarks, DrViews)
        m_featData.SetIntegerByName "WhichBody", m_whichBody
        Call m_feature.ModifyDefinition(m_featData, m_Part, m_modelComp)
    Else ' On Insert feature

        Dim paramNames, paramTypes, paramValues
        Dim pNames(0 To 0) As String
        Dim pTypes(0 To 0) As Long
        Dim pValues(0 To 0) As String
        Dim methods(0 To 8) As String
        methods(0) = m_macroPath
        methods(1) = "Macros"
        methods(2) = "swmMain"
        methods(3) = m_macroPath
        methods(4) = "Macros"
        methods(5) = "swmPM"
        methods(6) = ""
        methods(7) = ""
        methods(8) = ""
        Dim feat As Object
        pNames(0) = "WhichBody"
        pTypes(0) = swMacroFeatureParamTypeInteger
        pValues(0) = m_whichBody
    End If
End Sub
```

```

        paramNames = pNames
        paramTypes = pTypes
        paramValues = pValues
        Dim selBodies(0 To 0) As Body2
        Set selBodies(0) = m_bodySel
        Set feat = m_Part.FeatureManager.InsertMacroFeature3("AngleSurf", "", methods, (paramNames), (paramTypes), (paramValues), Nothing,
Nothing, selBodies, Nothing, swMacroFeatureByDefault)
    End If
End Sub

```

```

Public Sub OnCancel()
    If m_cmdState = swCmdEdit Then
        m_featData.ReleaseSelectionAccess
    End If
End Sub

```

```

Public Sub OnBodySelect()
    Dim selM
    Set selM = m_Part.SelectionManager
    Set m_bodySel = Nothing
    Set m_bodySel = selM.GetSelectedObject6(1, -1)
End Sub

```

```

Public Sub OnNumberChanged(Value As Long)
    m_whichBody = Value
End Sub

```

```

Public Function GetNumberValue()
    GetNumberValue = m_whichBody
End Function

```

[Back to top](#)

'PropMgrHdlr

```

Option Explicit
Implements PropertyManagerPage2Handler8

```

```

Dim m_pageObj As PropMgr

```

```

Public Sub Init(pageObj As PropMgr)
    Set m_pageObj = pageObj
End Sub

```

```

Private Sub PropertyManagerPage2Handler8_OnListboxRMBUp(ByVal Id As Long, ByVal PosX As Long, ByVal PosY As Long)
End Sub

```

```

Private Function PropertyManagerPage2Handler8_OnWindowFromHandleControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long
End Function

```

```

Private Sub PropertyManagerPage2Handler8_OnLostFocus(ByVal Id As Long)
End Sub

```

```

Private Sub PropertyManagerPage2Handler8_OnGainedFocus(ByVal Id As Long)
End Sub

```

```

Private Sub PropertyManagerPage2Handler8_OnPopupMenuItemUpdate(ByVal Id As Long, ByRef RetVal As Long)
End Sub

```

```

Private Sub PropertyManagerPage2Handler8_OnPopupMenuItem(ByVal Id As Long)
End Sub

```

```

Private Function PropertyManagerPage2Handler8_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal Id As Long)
As Boolean
End Function

```

```

Private Sub PropertyManagerPage2Handler8_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
End Sub

```

```

Private Sub PropertyManagerPage2Handler8_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)
End Sub

```

```

Private Function PropertyManagerPage2Handler8_OnActiveXControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long

```

```

End Function

Private Function PropertyManagerPage2Handler8_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long, ByRef
ItemText As String) As Boolean
    PropertyManagerPage2Handler8_OnSubmitSelection = True
End Function

Private Sub PropertyManagerPage2Handler8_OnSelectionboxCalloutDestroyed(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler8_OnSelectionboxCalloutCreated(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler8_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)
End Sub

Private Function PropertyManagerPage2Handler8_OnTabClicked(ByVal Id As Long) As Boolean
End Function

Private Sub PropertyManagerPage2Handler8_OnRedo()
End Sub

Private Sub PropertyManagerPage2Handler8_OnUndo()
End Sub

Private Sub PropertyManagerPage2Handler8_OnWhatsNew()
End Sub

Private Function PropertyManagerPage2Handler8_OnPreview() As Boolean
End Function

Private Function PropertyManagerPage2Handler8_OnNextPage() As Boolean
End Function

Private Function PropertyManagerPage2Handler8_OnPreviousPage() As Boolean
End Function

Private Sub PropertyManagerPage2Handler8_AfterActivation()
End Sub

Private Function PropertyManagerPage2Handler8_ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Long) As Boolean

End Function

Private Sub PropertyManagerPage2Handler8_OnButtonPress(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler8_OnClose(ByVal reason As Long)
    If reason = swPropertyManagerPageClose_Okay Then
        m_pageObj.GetCmd().OnOk
    ElseIf reason = swPropertyManagerPageClose_Cancel Then
        m_pageObj.GetCmd().OnCancel
    End If
End Sub

Private Sub PropertyManagerPage2Handler8_OnCheckboxCheck(ByVal Id As Long, ByVal Checked As Boolean)
End Sub

Private Sub PropertyManagerPage2Handler8_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
End Sub

Private Sub PropertyManagerPage2Handler8_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)
End Sub

Private Sub PropertyManagerPage2Handler8_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)
End Sub

```

```
Private Function PropertyManagerPage2Handler8_OnHelp() As Boolean
End Function
```

```
Private Sub PropertyManagerPage2Handler8_OnListBoxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
End Sub
```

```
Private Sub PropertyManagerPage2Handler8_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)
    m_page0bj.GetCmd().OnNumberChanged Int(Value)
End Sub
```

```
Private Sub PropertyManagerPage2Handler8_AfterClose()
    Set m_page0bj = Nothing
End Sub
```

```
Private Sub PropertyManagerPage2Handler8_OnOptionCheck(ByVal Id As Long)
End Sub
```

```
Private Sub PropertyManagerPage2Handler8_OnSelectionboxFocusChanged(ByVal Id As Long)
End Sub
```

```
Private Sub PropertyManagerPage2Handler8_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)
End Sub
```

```
Private Sub PropertyManagerPage2Handler8_OnSelectionBoxListChanged(ByVal Id As Long, ByVal Text As Long)
    m_page0bj.GetCmd().OnBodySelect
End Sub
```

[Back to top](#)

## Delete All Constraints in Selected Sketch Example (VBA)

This example shows how to delete all of the constraints in the selected sketch.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\surfaces\nozzle.sldprt.
' 2. Select Sketch14 in the FeatureManager design tree.
' 3. Examine the graphics area.
'
' Postconditions:
' 1. Deletes all constraints in Sketch14.
' 2. Examine the graphics area.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks

    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim vSketchSeg As Variant
    Dim swSketchSeg As SldWorks.SketchSegment
    Dim vSketchPt As Variant
    Dim swSketchPt As SldWorks.SketchPoint
    Dim swSelData As SldWorks.SelectData
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)
    Set swSketch = swFeat.GetSpecificFeature2

    ' Edit sketch
    swModel.EditSketch

    vSketchSeg = swSketch.GetSketchSegments
    For i = 0 To UBound(vSketchSeg)
        Set swSketchSeg = vSketchSeg(i)
        bRet = swSketchSeg.Select4(False, swSelData): Debug.Assert bRet
        swModel.SketchConstraintsDeleteAll
    Next i

    vSketchPt = swSketch.GetSketchPoints2
    For i = 0 To UBound(vSketchPt)
        Set swSketchPt = vSketchPt(i)
        bRet = swSketchPt.Select4(False, swSelData): Debug.Assert bRet
        swModel.SketchConstraintsDeleteAll
    Next i

    ' Exit sketch
    swModel.InsertSketch2 True

End Sub
```

### Delete All Decals Example (C#)

This example shows how to delete all decals in a model.

```
//-----  
// Preconditions: Open a part with one or more decals.  
//  
// Postconditions: Deletes all of the decals from the part.  
// -----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace DeleteAllDecals_CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            ModelDocExtension swModelDocExt;  
            Boolean boolStatus;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            boolStatus = swModelDocExt.DeleteAllDecals();  
  
        }  
  
        public SldWorks swApp;  
    }  
}
```



### Delete All Decals Example (VBA)

This example shows how to delete all decals from a model.

```
'-----  
' Preconditions: Open a part with one or more decals.  
'  
' Postconditions: Deletes all of the decals from the part.  
'-----  
  
Option Explicit  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim boolStatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    boolStatus = swModelDocExt.DeleteAllDecals()  
  
End Sub
```

### Delete All Decals Example (VB.NET)

This example shows how to delete all decals in a model.

```
'-----  
' Preconditions: Open a part with one or more decals.  
'-----  
' Postconditions: Deletes all of the decals from the part.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim boolStatus As Boolean
```

```
        swModel = swApp.ActiveDoc
```

```
        swModelDocExt = swModel.Extension
```

```
        boolStatus = swModelDocExt.DeleteAllDecals()
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

## Delete and Insert Instances in Variable Pattern Feature Example (C#)

This example shows how to delete and insert instances in a variable pattern feature.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\bottle.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects the variable pattern feature.
// 2. Rolls back and accesses the variable pattern feature.
//    a. Deletes the last instance of the variable pattern in the table.
//    b. Deletes a dimension in the table.
//    c. Gets the names of the controlling dimensions.
//    d. Inserts a new instance in the table.
//    e. Rolls forward the variable pattern feature.
//    NOTE: It can take several minutes for this step to complete.
// 3. Selects the variable pattern again.
//    a. Rolls back and accesses the variable pattern feature.
//    b. Sets new values for the new instance.
//    c. Rolls forward the variable pattern feature.
// 4. Examine the Immediate window and graphics area.
//
// NOTE: Because the part is used elsewhere, do not save change.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            DimPatternFeatureData swDimensionDrivenTablePatternFeat = default(DimPatternFeatureData);
            bool status = false;
            string DimensionName = null;
            string errorString = null;
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            status = swModelDocExt.SelectByID2("VarPattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelMgr.GetSelectedObjects(1, -1);
            swDimensionDrivenTablePatternFeat = (DimPatternFeatureData)swFeature.GetDefinition();

            //Roll back and access the variable pattern feature
            status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, null);

            //Delete the last instance of the variable pattern in the table
            status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(-1);
            Debug.Print("Last instance of variable pattern deleted from table? " + status);

            //Delete a dimension in the table
            DimensionName = "D3@Fillet1";
            status = swDimensionDrivenTablePatternFeat.DeleteDimension(DimensionName);
            Debug.Print("D3@Fillet1 dimension deleted from table? " + status);

            //Get the names of the controlling dimensions
            int nbr = 0;
            int dimNbr = 0;

            string controllingDimName = null;
            string[] controllingDimNames = null;
            nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount();
            Debug.Print("Number of pattern instances: " + nbr);
            Debug.Print(" Controlling dimension names: ");
            dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount();
            controllingDimNames = new string[dimNbr + 1];
            for (i = 0; i < dimNbr; i++)
            {
                controllingDimNames[i] = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i);
                controllingDimName = controllingDimNames[i];
                Debug.Print(" " + controllingDimName);
            }

            //Insert instance in table
            status = swDimensionDrivenTablePatternFeat.AddInstanceAt(false, -1);
            Debug.Print("Instance added to table? " + status);
            nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount();
            Debug.Print("Number of pattern instances: " + nbr);

            //Roll forward the variable pattern feature
            //NOTE: It can take several minutes for this step to complete.
            swFeature.ModifyDefinition(swDimensionDrivenTablePatternFeat, swModel, null);

            //Select the variable pattern again
            status = swModelDocExt.SelectByID2("VarPattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelMgr.GetSelectedObjects(1, -1);
            swDimensionDrivenTablePatternFeat = (DimPatternFeatureData)swFeature.GetDefinition();

            //Roll back and access the variable pattern feature
            status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, null);

            //Set new values for the new instance
            errorString = swDimensionDrivenTablePatternFeat.SetInstanceDimensionValue(nbr, "D3@Sketch2@bottle.moPart_c", 0.15);
            if (string.IsNullOrEmpty(errorString))
            {
                Debug.Print("Set new value for new instance[" + nbr + "] D3@Sketch2 dimension? True");
            }
        }
    }
}
```

```

        else
        {
            Debug.Print("Set new value for new instance[" + nbr + "] D3@Sketch2 dimension? " + errorString);
        }
        errorString = swDimensionDrivenTablePatternFeat.SetInstanceDimensionValue(nbr, "D1@Fillet1@bottle.moPart_c", 0.01);
        if (string.IsNullOrEmpty(errorString))
        {
            Debug.Print("Set new value for new instance[" + nbr + "] D2Fillet2 dimension? True");
        }
        else
        {
            Debug.Print("Set new value for new instance[" + nbr + "] D2Fillet2 dimension? " + errorString);
        }

        //Roll forward the variable pattern feature
        swFeature.ModifyDefinition(swDimensionDrivenTablePatternFeat, swModel, null);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

## Delete and Insert Instances in Variable Pattern Feature Example (VBA)

This example shows how to delete and insert instances in a variable pattern feature.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\bottle.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects the variable pattern feature.
' 2. Rolls back and accesses the variable pattern feature.
'   a. Deletes the last instance of the variable pattern in the table.
'   b. Deletes a dimension in the table.
'   c. Gets the names of the controlling dimensions.
'   d. Inserts a new instance in the table.
'   e. Rolls forward the variable pattern feature.
'   NOTE: It can take several minutes for this step to complete.
' 3. Selects the variable pattern again.
'   a. Rolls back and accesses the variable pattern feature.
'   b. Sets new values for the new instance.
'   c. Rolls forward the variable pattern feature.
' 4. Examine the Immediate window and graphics area.
'
' NOTE: Because the part is used elsewhere, do not save change.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swDimensionDrivenTablePatternFeat As SldWorks.DimPatternFeatureData
Dim status As Boolean
Dim DimensionName As String
Dim errorString As String
Dim i As Long

Sub main()
    Set swApp = Application.SldWorks

    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    status = swModelDocExt.SelectByID2("VarPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swDimensionDrivenTablePatternFeat = swFeature.GetDefinition

    'Roll back and access the variable pattern feature
    status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

    'Delete the last instance of the variable pattern in the table
    status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(-1)
    Debug.Print "Last instance of variable pattern deleted from table? " & status

    'Delete a dimension in the table
    DimensionName = "D3@Fillet1"
    status = swDimensionDrivenTablePatternFeat.DeleteDimension(DimensionName)
    Debug.Print "D3@Fillet1 dimension deleted from table? " & status

    'Get the names of the controlling dimensions
    Dim nbr As Long
    Dim dimNbr As Long
    Dim controllingDimNames() As String
    Dim controllingDimName As String
    nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount
    Debug.Print ("Number of pattern instances: " & nbr)
    Debug.Print (" Controlling dimension names: ")
    dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount
    ReDim Preserve controllingDimNames(dimNbr)
    For i = 0 To dimNbr - 1
        controllingDimNames(i) = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i)
        controllingDimName = controllingDimNames(i)
        Debug.Print (" " & controllingDimName)
    Next i

    'Insert instance in table
    status = swDimensionDrivenTablePatternFeat.AddInstanceAt(False, -1)
    Debug.Print "Instance added to table? " & status
    nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount
    Debug.Print ("Number of pattern instances: " & nbr)

    'Roll forward the variable pattern feature
    'NOTE: It can several minutes for this step to complete.
    swFeature.ModifyDefinition swDimensionDrivenTablePatternFeat, swModel, Nothing

    'Select the variable pattern again
    status = swModelDocExt.SelectByID2("VarPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swDimensionDrivenTablePatternFeat = swFeature.GetDefinition

    'Roll back and access the variable pattern feature
    status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

    'Set new values for the new instance
    errorString = swDimensionDrivenTablePatternFeat.SetInstanceDimensionValue(nbr, "D3@Sketch2@bottle.moPart_c", 0.15)
```

```

If errorString = "" Then
    Debug.Print "Set new value for new instance[" & nbr & "] D3@Sketch2 dimension? True"
Else
    Debug.Print "Set new value for new instance[" & nbr & "] D3@Sketch2 dimension? " & errorString
End If
errorString = swDimensionDrivenTablePatternFeat.SetInstanceDimensionValue(nbr, "D1@Fillet1@bottle.moPart_c", 0.01)
If errorString = "" Then
    Debug.Print "Set new value for new instance[" & nbr & "] D2Fillet2 dimension? True"
Else
    Debug.Print "Set new value for new instance[" & nbr & "] D2Fillet2 dimension? " & errorString
End If

'Roll forward the variable pattern feature
swFeature.ModifyDefinition swDimensionDrivenTablePatternFeat, swModel, Nothing

End Sub

```

## Delete and Insert Instances in Variable Pattern Feature Example (VB.NET)

This example shows how to delete and insert instances in a variable pattern feature.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\bottle.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects the variable pattern feature.
' 2. Rolls back and accesses the variable pattern feature.
'   a. Deletes the last instance of the variable pattern in the table.
'   b. Deletes a dimension in the table.
'   c. Gets the names of the controlling dimensions.
'   d. Inserts a new instance in the table.
'   e. Rolls forward the variable pattern feature.
'   NOTE: It can take several minutes for this step to complete.
' 3. Selects the variable pattern again.
'   a. Rolls back and accesses the variable pattern feature.
'   b. Sets new values for the new instance.
'   c. Rolls forward the variable pattern feature.
' 4. Examine the Immediate window and graphics area.
'
' NOTE: Because the part is used elsewhere, do not save change.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swDimensionDrivenTablePatternFeat As DimPatternFeatureData
        Dim status As Boolean
        Dim DimensionName As String
        Dim errorString As String
        Dim i As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        status = swModelDocExt.SelectByID("VarPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swDimensionDrivenTablePatternFeat = swFeature.GetDefinition

        'Roll back and access the variable pattern feature
        status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

        'Delete the last instance of the variable pattern in the table
        status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(-1)
        Debug.Print("Last instance of variable pattern deleted from table? " & status)

        'Delete a dimension in the table
        DimensionName = "D3@Fillet1"
        status = swDimensionDrivenTablePatternFeat.DeleteDimension(DimensionName)
        Debug.Print("D3@Fillet1 dimension deleted from table? " & status)

        'Get the names of the controlling dimensions
        Dim nbr As Integer
        Dim dimNbr As Integer
        Dim controllingDimNames() As String
        Dim controllingDimName As String
        nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount
        Debug.Print("Number of pattern instances: " & nbr)
        Debug.Print(" Controlling dimension names: ")
        dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount
        ReDim Preserve controllingDimNames(dimNbr)
        For i = 0 To dimNbr - 1
            controllingDimNames(i) = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i)
            controllingDimName = controllingDimNames(i)
            Debug.Print(" " & controllingDimName)
        Next i

        'Insert instance in table
        status = swDimensionDrivenTablePatternFeat.AddInstanceAt(False, -1)
        Debug.Print("Instance added to table? " & status)
        nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount
        Debug.Print("Number of pattern instances: " & nbr)

        'Roll forward the variable pattern feature
        'NOTE: It can several minutes for this step to complete.
        swFeature.ModifyDefinition(swDimensionDrivenTablePatternFeat, swModel, Nothing)

        'Select the variable pattern again
        status = swModelDocExt.SelectByID("VarPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swDimensionDrivenTablePatternFeat = swFeature.GetDefinition

        'Roll back and access the variable pattern feature
        status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

        'Set new values for the new instance
        errorString = swDimensionDrivenTablePatternFeat.SetInstanceDimensionValue(nbr, "D3@Sketch2@bottle.moPart_c", 0.15)
        If errorString = "" Then
    
```

```

        Debug.Print("Set new value for new instance[" & nbr & "] D3@Sketch2 dimension? True")
    Else
        Debug.Print("Set new value for new instance[" & nbr & "] D3@Sketch2 dimension? " & errorString)
    End If
    errorString = swDimensionDrivenTablePatternFeat.SetInstanceDimensionValue(nbr, "D1@Fillet1@bottle.moPart_c", 0.01)
    If errorString = "" Then
        Debug.Print("Set new value for new instance[" & nbr & "] D2Fillet2 dimension? True")
    Else
        Debug.Print("Set new value for new instance[" & nbr & "] D2Fillet2 dimension? " & errorString)
    End If

    'Roll forward the variable pattern feature
    swFeature.ModifyDefinition(swDimensionDrivenTablePatternFeat, swModel, Nothing)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```



## Delete Attribute Example (C#)

This example shows how to delete the selected attribute.

```
//-----
// Preconditions:
// 1. Verify that the specified part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Selects Cut-Extrude3.
// 3. Adds attribute att 1 to the selected feature.
// 4. Rebuilds the part.
// 5. Deletes attribute att 1.
// 6. Expand Cut-Extrude3 in the FeatureManager design tree
//    and examine the Immediate window to verify step 5.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DeleteAttributeCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            //1 = invisible
            //0 = visible
            const int CreateVisible = 0;
            const string AttDefName = "XML_string";
            const string AttLenDimName = "att_name";
            const string AttLenValueName = "att_value";
            const string AttRootName = "att";
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = null;
            Entity swEnt = default(Entity);
            AttributeDef swAttDef = default(AttributeDef);
            SolidWorks.Interop.sldworks.Attribute swAtt = null;
            Parameter swParamName = default(Parameter);
            Parameter swParamValue = default(Parameter);
            string AttName = "";
            int i = 0;
            bool bRet = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\smartcomponents\\bearing.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Select feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swModelDocExt.SelectByID2("Cut-Extrude3", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swEnt = (Entity)swFeat;

            //Set attribute att 1
            swAttDef = (AttributeDef)swApp.DefineAttribute(AttDefName);
            bRet = swAttDef.AddParameter(AttLenDimName, (int)swParamType_e.swParamTypeString, 0.0, 0);
            bRet = swAttDef.AddParameter(AttLenValueName, (int)swParamType_e.swParamTypeDouble, 0.0, 0);
            bRet = swAttDef.Register();
            while (swAtt == null)
            {
                // Get a unique attribute name
                i = i + 1;
                AttName = AttRootName + " " + i.ToString();
                swAtt = (SolidWorks.Interop.sldworks.Attribute)swAttDef.CreateInstance5(swModel, swEnt, AttName, CreateVisible, (int)swInConfigurationOpt
s_e.swThisConfiguration);
            }
            swParamName = (Parameter)swAtt.GetParameter(AttLenDimName);
            swParamValue = (Parameter)swAtt.GetParameter(AttLenValueName);
            bRet = swParamName.SetString2(AttName + " - " + AttLenDimName, (int)swInConfigurationOpts_e.swAllConfiguration, "");
            bRet = swParamValue.SetDoubleValue2(i * 10, (int)swInConfigurationOpts_e.swAllConfiguration, "");
            if ((swAtt != null))
            {
                Debug.Print("Attribute " + AttName + " created.");
            }
            else
            {
                Debug.Print("Attribute " + AttName + " not created.");
            }
        }

        swModel.ForceRebuild3(true);

        //Delete attribute att 1
        bRet = swModelDocExt.SelectByID2("att 1", "ATTRIBUTE", 0, 0, 0, false, 0, null, 0);
        swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
        swAtt = (SolidWorks.Interop.sldworks.Attribute)swFeat.GetSpecificFeature2();
        AttName = swAtt.GetName();
        bRet = swAtt.Delete(true);
        if (bRet)
        {
            Debug.Print("Attribute " + AttName + " deleted.");
        }
        else
        {
            Debug.Print("Attribute " + AttName + " not deleted.");
        }
    }
}
```

```
    }  
    /// <summary>  
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

## Delete Attribute Example (VBA)

This example shows how to delete the selected attribute.

```

'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects Cut-Extrude3.
' 3. Adds attribute att 1 to the selected feature.
' 4. Rebuilds the part.
' 5. Deletes attribute att 1.
' 6. Expand Cut-Extrude3 in the FeatureManager design tree
'    and examine the Immediate window to verify step 5.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    '1 = invisible
    '0 = visible
    Const CreateVisible As Long = 0
    Const AttDefName As String = "XML_string"
    Const AttLenDimName As String = "att_name"
    Const AttLenValueName As String = "att_value"
    Const AttRootName As String = "att"
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swEnt As SldWorks.Entity
    Dim swAttDef As SldWorks.AttributeDef
    Dim swAtt As SldWorks.Attribute
    Dim swParamName As SldWorks.Parameter
    Dim swParamValue As SldWorks.Parameter
    Dim AttName As String
    Dim i As Long
    Dim bRet As Boolean
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = CreateObject("SldWorks.Application")

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\bearing.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select feature
    Set swModelDocExt = swModel.Extension
    bRet = swModelDocExt.SelectByID2("Cut-Extrude3", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swEnt = swFeat

    'Set attribute att 1
    Set swAttDef = swApp.DefineAttribute(AttDefName)
    bRet = swAttDef.AddParameter(AttLenDimName, swParamTypeString, 0#, 0)
    bRet = swAttDef.AddParameter(AttLenValueName, swParamTypeDouble, 0#, 0)
    bRet = swAttDef.Register
    While swAtt Is Nothing
        ' Get a unique attribute name
        i = i + 1
        AttName = AttRootName + Str(i)
        Set swAtt = swAttDef.CreateInstance5(swModel, swEnt, AttName, CreateVisible, swThisConfiguration)
    Wend
    Set swParamName = swAtt.GetParameter(AttLenDimName)
    Set swParamValue = swAtt.GetParameter(AttLenValueName)
    bRet = swParamName.SetString2(AttName & " - " & AttLenDimName, swAllConfiguration, "")
    bRet = swParamValue.SetDouble2(i * 10, swAllConfiguration, "")
    If Not swAtt Is Nothing Then
        Debug.Print "Attribute " & AttName & " created."
    Else
        Debug.Print "Attribute " & AttName & " not created."
    End If

    swModel.ForceRebuild3 True

    'Delete attribute att 1
    bRet = swModelDocExt.SelectByID2("att 1", "ATTRIBUTE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swAtt = swFeat.GetSpecificFeature2
    AttName = swAtt.GetName
    bRet = swAtt.Delete(True)
    If bRet Then
        Debug.Print "Attribute " & AttName & " deleted."
    Else

```

```
        Debug.Print "Attribute " & AttName & " not deleted."  
    End If
```

```
End Sub
```

## Delete Attribute Example (VB.NET)

This example shows how to delete the selected attribute.

```

-----
Preconditions:
1. Verify that the specified part document to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified part document.
2. Selects Cut-Extrude3.
3. Adds attribute att 1 to the selected feature.
4. Rebuilds the part.
5. Deletes attribute att 1.
6. Expand Cut-Extrude3 in the FeatureManager design tree
   and examine the Immediate window to verify step 5.

NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        '1 = invisible
        '0 = visible
        Const CreateVisible As Integer = 0
        Const AttDefName As String = "XML_string"
        Const AttLenDimName As String = "att_name"
        Const AttLenValueName As String = "att_value"
        Const AttRootName As String = "att"
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Object
        Dim swEnt As Entity
        Dim swAttDef As AttributeDef
        Dim swAtt As SolidWorks.Interop.sldworks.Attribute = Nothing
        Dim swParamName As Parameter
        Dim swParamValue As Parameter
        Dim AttName As String = ""
        Dim i As Integer
        Dim bRet As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        'Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\smartcomponents\bearing.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select feature
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SelectByID2("Cut-Extrude3", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swEnt = swFeat

        'Set attribute att 1
        swAttDef = swApp.DefineAttribute(AttDefName)
        bRet = swAttDef.AddParameter(AttLenDimName, swParamType_e.swParamTypeString, 0.0#, 0)
        bRet = swAttDef.AddParameter(AttLenValueName, swParamType_e.swParamTypeDouble, 0.0#, 0)
        bRet = swAttDef.Register
        While swAtt Is Nothing
            ' Get a unique attribute name
            i = i + 1
            AttName = AttRootName + Str(i)
            swAtt = swAttDef.CreateInstance5(swModel, swEnt, AttName, CreateVisible, swInConfigurationOpts_e.swThisConfiguration)
        End While
        swParamName = swAtt.GetParameter(AttLenDimName)
        swParamValue = swAtt.GetParameter(AttLenValueName)
        bRet = swParamName.SetString2(AttName & " - " & AttLenDimName, swInConfigurationOpts_e.swAllConfiguration, "")
        bRet = swParamValue.SetDouble2(i * 10, swInConfigurationOpts_e.swAllConfiguration, "")
        If Not swAtt Is Nothing Then
            Debug.Print("Attribute " & AttName & " created.")
        Else
            Debug.Print("Attribute " & AttName & " not created.")
        End If

        swModel.ForceRebuild3(True)

        'Delete attribute att 1
        bRet = swModelDocExt.SelectByID2("att 1", "ATTRIBUTE", 0, 0, 0, False, 0, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swAtt = swFeat.GetSpecificFeature2
        AttName = swAtt.GetName
        bRet = swAtt.Delete(True)
        If bRet Then
            Debug.Print("Attribute " & AttName & " deleted.")
        Else
            Debug.Print("Attribute " & AttName & " not deleted.")
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

## Delete Blended Faces Example (C#)

This example shows how to delete blended faces.

**NOTE:** You can only delete blended faces from a temporary body.

```

//-----
// Preconditions:
// 1. Open a part that contains one solid body
// and at least one blended (fillet) face.
// 2. Select a blended face.
//
// Postconditions:
// 1. Creates a new part, which has the same body as
// the original part, but the selected blended
// face is removed.
// 2. Examine the graphics area.
//
// NOTE: It might not be possible to remove the
// selected blended face. If it's not removed, then
// the new body is the same as the original
// body.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            // 1 = Invisible
            // 0 = Visible
            const int CreateVisible = 0;

            const string sAttDefName = "temp_attrib";
            const string sAttRootName = "temp";

            AttributeDef swAttDef = default(AttributeDef);
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            PartDoc swPart = default(PartDoc);
            int nSelCount = 0;
            Face2 swFace = default(Face2);
            Entity swEnt = default(Entity);
            SolidWorks.Interop.sldworks.Attribute swAtt = null;
            ArrayList vAtt = new ArrayList();
            ArrayList vFaceArr = new ArrayList();
            Feature swFeat = default(Feature);
            object[] vBodies = null;
            Body2 swBody = default(Body2);
            Body2 swCopyBody = default(Body2);
            PartDoc swNewPart = default(PartDoc);
            int i = 0;
            bool bRet = false;

            swAttDef = (AttributeDef)swApp.DefineAttribute(sAttDefName);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swPart = (PartDoc)swModel;
            bRet = swAttDef.Register();

            // Add attribute to selected faces
            nSelCount = swSelMgr.GetSelectedObjectCount2(-1);
            for (i = 1; i <= nSelCount; i++)
            {
                swFace = (Face2)swSelMgr.GetSelectedObject6(i, -1);
                swEnt = (Entity)swFace;
                swAtt = (SolidWorks.Interop.sldworks.Attribute)swAttDef.CreateInstance5(swModel, swEnt, sAttRootName + i, CreateVisible, (int)swInConfigur
                ationOpts_e.swAllConfiguration);
                vAtt.Add(swAtt);
            }
            vBodies = (object[])swPart.GetBodies2((int)swBodyType_e.swAllBodies, false);
            swBody = vBodies[0] as Body2;
            swCopyBody = (Body2)swBody.Copy();

            // Remove attribute from faces
            for (i = 1; i <= nSelCount; i++)
            {
                swAtt = (SolidWorks.Interop.sldworks.Attribute)vAtt[i - 1];
                bRet = swAtt.Delete(true);
            }
            vFaceArr = (ArrayList)GetFacesWithAttribute(swApp, swCopyBody, swAttDef);
            Face2[] aFaceArr = (Face2[])vFaceArr.ToArray(typeof(Face2));

            // Can only delete blends from a temporary body
            Debug.Assert(swCopyBody.IsTemporaryBody());

            bRet = swCopyBody.DeleteBlends3(aFaceArr, true, true);
            swNewPart = (PartDoc)swApp.NewPart();
            swFeat = (Feature)swNewPart.CreateFeatureFromBody3(swCopyBody, false, (int)swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck);
        }
        public object GetFacesWithAttribute(SldWorks swApp, Body2 swBody, AttributeDef swAttDef)
        {
            Face2 swFace = default(Face2);
            Entity swEnt = default(Entity);
            SolidWorks.Interop.sldworks.Attribute swAttCopy = default(SolidWorks.Interop.sldworks.Attribute);
            ArrayList swFaceArr = new ArrayList();

            // Search for faces on temporary body based
            // on copied attributes
            swFace = (Face2)swBody.GetFirstFace();
            while ((null != swFace))
            {
                swEnt = (Entity)swFace;
                swAttCopy = null;
            }
        }
    }
}

```

```

        // Only one instance of attribute should exist
        swAttCopy = (SolidWorks.Interop.sldworks.Attribute)swEnt.FindAttribute(swAttDef, 0);
        if ((swAttCopy != null))
        {
            swFaceArr.Add(swFace);
        }
        swFace = (Face2)swFace.GetNextFace();
    }

    return swFaceArr;
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

## Delete Blended Faces Example (VBA)

This example shows how to delete blended faces.

**NOTE:** You can only delete blended faces from a temporary body.

```

'-----
' Preconditions:
' 1. Open a part that contains one solid body
'   and at least one blended (fillet) face.
' 2. Select a blended face.
'
' Postconditions:
' 1. Creates a new part, which has the same body as
'   the original part, but the selected blended
'   face is removed.
' 2. Examine the graphics area.
'
' NOTE: It might not be possible to remove the
' selected blended face. If it's not removed, then
' the new body is the same as the original
' body.
'-----

```

Option Explicit

Function GetFacesWithAttribute(swApp As SldWorks.SldWorks, swBody As SldWorks.Body2, swAttDef As SldWorks.AttributeDef) As Variant

```

    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swAttCopy As SldWorks.Attribute
    Dim swFaceArr() As SldWorks.Face2

    ' Search for faces on temporary body based
    ' on copied attributes
    ReDim swFaceArr(0)
    Set swFace = swBody.GetFirstFace
    Do While Not Nothing Is swFace
        Set swEnt = swFace
        Set swAttCopy = Nothing
        ' Only one instance of attribute should exist
        Set swAttCopy = swEnt.FindAttribute(swAttDef, 0)
        If Not swAttCopy Is Nothing Then
            Set swFaceArr(UBound(swFaceArr)) = swFace
            ReDim Preserve swFaceArr(UBound(swFaceArr) + 1)
        End If
        Set swFace = swFace.GetNextFace
    Loop
    ReDim Preserve swFaceArr(UBound(swFaceArr) - 1)
    GetFacesWithAttribute = swFaceArr
End Function

```

Sub main()

```

    '1 = Invisible
    '0 = Visible
    Const CreateVisible As Long = 0

    Const sAttDefName As String = "temp_attrib"
    Const sAttRootName As String = "temp"

    Dim swApp As SldWorks.SldWorks
    Dim swAttDef As SldWorks.AttributeDef
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swPart As SldWorks.PartDoc
    Dim nSelCount As Long
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swAtt() As SldWorks.Attribute
    Dim vFaceArr As Variant
    Dim swFeat As SldWorks.Feature
    Dim vBodies As Variant
    Dim swBody As SldWorks.Body2
    Dim swCopyBody As SldWorks.Body2
    Dim swNewPart As SldWorks.PartDoc
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swAttDef = swApp.DefineAttribute(sAttDefName)
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swPart = swModel

    bRet = swAttDef.Register

    ' Add attribute to selected faces
    nSelCount = swSelMgr.GetSelectedObjectCount
    ReDim swAtt(nSelCount)
    For i = 1 To nSelCount
        Set swFace = swSelMgr.GetSelectedObject6(i, -1)
        Set swEnt = swFace
        Set swAtt(i - 1) = swAttDef.CreateInstance5(swModel, swEnt, sAttRootName & i, CreateVisible, swAllConfiguration)
    Next i

    vBodies = swPart.GetBodies2(swAllBodies, False)
    Set swBody = vBodies(0)
    Set swCopyBody = swBody.Copy

```



```
' Remove attribute from faces
For i = 1 To nSelCount
    bRet = swAtt(i - 1).Delete(True)
Next i

vFaceArr = GetFacesWithAttribute(swApp, swCopyBody, swAttDef)

' Can only delete blends from a temporary body
Debug.Assert swCopyBody.IsTemporaryBody

bRet = swCopyBody.DeleteBlends3(vFaceArr, True, True)

Set swNewPart = swApp.NewPart
Set swFeat = swNewPart.CreateFeatureFromBody3(swCopyBody, False, swCreateFeatureBodyCheck)

End Sub
```

## Delete Blended Faces Example (VB.NET)

This example shows how to delete blended faces.

**NOTE:** You can only delete blended faces from a temporary body.

```

'-----
' Preconditions:
' 1. Open a part that contains one solid body
'   and at least one blended (fillet) face.
' 2. Select a blended face.
'
' Postconditions:
' 1. Creates a new part, which has the same body as
'   the original part, but the selected blended
'   face is removed.
' 2. Examine the graphics area.
'
' NOTE: It might not be possible to remove the
' selected blended face. If it's not removed, then
' the new body is the same as the original
' body.
'-----
Option Explicit On

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        ' 1 = Invisible
        ' 0 = Visible
        Const CreateVisible As Integer = 0

        Const sAttDefName As String = "temp_attrib"
        Const sAttRootName As String = "temp"

        Dim swAttDef As AttributeDef
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swPart As PartDoc
        Dim nSelCount As Integer
        Dim swFace As Face2
        Dim swEnt As Entity
        Dim swAtt() As SolidWorks.Interop.sldworks.Attribute
        Dim vFaceArr As Object
        Dim swFeat As Feature
        Dim vBodies As Object
        Dim swBody As Body2
        Dim swCopyBody As Body2
        Dim swNewPart As PartDoc
        Dim i As Integer
        Dim bRet As Boolean

        swAttDef = swApp.DefineAttribute(sAttDefName)
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swPart = swModel
        bRet = swAttDef.Register

        ' Add attribute to selected faces
        nSelCount = swSelMgr.GetSelectedObjectCount
        ReDim swAtt(nSelCount)
        For i = 1 To nSelCount
            swFace = swSelMgr.GetSelectedObject6(i, -1)
            swEnt = swFace
            swAtt(i - 1) = swAttDef.CreateInstance5(swModel, swEnt, sAttRootName & i, CreateVisible, swInConfigurationOpts_e.swAllConfiguration)
        Next i
        vBodies = swPart.GetBodies2(swBodyType_e.swAllBodies, False)
        swBody = vBodies(0)
        swCopyBody = swBody.Copy

        ' Remove attribute from faces
        For i = 1 To nSelCount
            bRet = swAtt(i - 1).Delete(True)
        Next i
        vFaceArr = GetFacesWithAttribute(swApp, swCopyBody, swAttDef)

        ' Can only delete blends from a temporary body
        Debug.Assert(swCopyBody.IsTemporaryBody)

        bRet = swCopyBody.DeleteBlends3(vFaceArr, True, True)
        swNewPart = swApp.NewPart
        swFeat = swNewPart.CreateFeatureFromBody3(swCopyBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck)

    End Sub

    Public Function GetFacesWithAttribute(ByVal swApp As SldWorks, ByVal swBody As Body2, ByVal swAttDef As AttributeDef) As Object
        Dim swFace As Face2
        Dim swEnt As Entity
        Dim swAttCopy As SolidWorks.Interop.sldworks.Attribute
        Dim swFaceArr() As Face2

        ' Search for faces on temporary body based on copied attributes
        ReDim swFaceArr(0)
        swFace = swBody.GetFirstFace
        Do While Not Nothing Is swFace
            swEnt = swFace
            swAttCopy = Nothing

            ' Only one instance of attribute should exist
            swAttCopy = swEnt.FindAttribute(swAttDef, 0)
            If Not swAttCopy Is Nothing Then
                swFaceArr(UBound(swFaceArr)) = swFace
                ReDim Preserve swFaceArr(UBound(swFaceArr) + 1)
            End If
            swFace = swFace.NextFace
        Loop
    End Function

```

```
        End If
        swFace = swFace.GetNextFace
    Loop
    ReDim Preserve swFaceArr(UBound(swFaceArr) - 1)
    GetFacesWithAttribute = swFaceArr
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

Public swApp As SldWorks

End Class
```

## Delete Decal Example (C#)

This example shows how to delete a decal from a model.

```
//-----
// Preconditions:
// 1. Open a model document that has at least one
//    decal applied to it.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Deletes all of the decals.
// 2. Examine the Immediate window.
//-----
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DeleteDecal_CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            Object[] arrDecals;
            Decal swDecal;
            Boolean boolstatus;
            int iDecalCnt;
            int iDecalID;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            iDecalCnt = swModelDocExt.GetDecalsCount();
            Debug.Print("Decal Count : " + iDecalCnt);

            arrDecals = (Object[])swModelDocExt.GetDecals();

            for (int i = 0; i <= arrDecals.Length - 1; i++)
            {
                swDecal = (Decal)arrDecals[i];
                iDecalID = swDecal.DecalID;
                Debug.Print("Decal ID: " + iDecalID);
                boolstatus = swModelDocExt.DeleteDecal(iDecalID, true);
            }
        }

        public SldWorks swApp;
    }
}
```

### Delete Decal Example (VBA)

This example shows how to delete a decal from a model.

```
'-----  
' Preconditions:  
' 1. Open a model document that has at least one  
'    decal applied to it.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Deletes all of the decals.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim arrDecals As Variant  
Dim swDecal As SldWorks.Decal  
Dim boolstatus As Boolean  
Dim iDecalCnt As Integer  
Dim iDecalID As Integer  
Dim i As Integer  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    iDecalCnt = swModelDocExt.GetDecalsCount  
    Debug.Print "Decal Count : " & iDecalCnt  
  
    arrDecals = swModelDocExt.GetDecals()  
  
    For i = LBound(arrDecals) To UBound(arrDecals)  
        Set swDecal = arrDecals(i)  
        iDecalID = swDecal.DecalID  
        Debug.Print "Decal ID : " & iDecalID  
        boolstatus = swModelDocExt.DeleteDecal(iDecalID, True)  
    Next  
End Sub
```

## Delete Decal Example (VB.NET)

This example shows how to delete a decal from a model.

```
'-----  
' Preconditions:  
' 1. Open a model document that has at least one  
'    decal applied to it.  
' 2. Open an Immediate window.  
'-----  
' Postconditions:  
' 1. Deletes all of the decals.  
' 2. Examine the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim arrDecals As Object  
        Dim swDecal As Decal  
        Dim boolstatus As Boolean  
        Dim iDecalCnt As Integer  
        Dim iDecalID As Integer  
        Dim i As Integer  
  
        swModel = swApp.ActiveDoc  
        swModelDocExt = swModel.Extension  
  
        iDecalCnt = swModelDocExt.GetDecalsCount  
        Debug.Print("Decal Count : " & iDecalCnt)  
  
        arrDecals = swModelDocExt.GetDecals()  
  
        For i = LBound(arrDecals) To UBound(arrDecals)  
            swDecal = arrDecals(i)  
            iDecalID = swDecal.DecalID  
            Debug.Print("Decal ID : " & iDecalID)  
            boolstatus = swModelDocExt.DeleteDecal(iDecalID, True)  
        Next  
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

### Delete Design Table Example (VBA)

This example shows how to delete a design table.

```
'-----  
' Preconditions:  
' 1. Part or assembly document is open that contains a  
'    design table.  
' 2. Verify that a design table exists by expanding  
'    Tables in the ConfigurationManager.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Deletes the design table.  
' 2. Examine the Immediate window and ConfigurationManager.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim status As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.HasDesignTable  
    If status Then  
        swModel.DeleteDesignTable  
        Debug.Print "Design table deleted."  
    Else  
        Debug.Print "Model document does not contain design table."  
    End If  
End Sub
```

## Delete Faces Example (VBA)

This example shows how to delete faces.

**NOTE:** You can only delete faces from a temporary body.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\block.sldprt.
' 2. Select a cylindrical face.
'
' Postconditions:
' 1. Creates a new part, which contains the same body as
'    the original part but with the selected face removed.
' 2. Examine the graphics area.
'
' NOTE: It might not be possible to remove the
' selected face. If this is the case, then
' the new body is the same as the original
' body.
'-----
```

Option Explicit

```
Function GetFacesWithAttribute(swApp As SldWorks.SldWorks, swBody As SldWorks.Body2, swAttDef As SldWorks.AttributeDef) As Variant
```

```
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swAttCopy As SldWorks.Attribute
    Dim swFaceArr() As SldWorks.Face2

    ' Search for faces on temporary body based on copied attributes
    ReDim swFaceArr(0)
    Set swFace = swBody.GetFirstFace
    Do While Not Nothing Is swFace
        Set swEnt = swFace
        Set swAttCopy = Nothing
        ' Only one instance of attribute on a face should exist
        Set swAttCopy = swEnt.FindAttribute(swAttDef, 0)
        If Not swAttCopy Is Nothing Then
            Set swFaceArr(UBound(swFaceArr)) = swFace
            ReDim Preserve swFaceArr(UBound(swFaceArr) + 1)
        End If
        Set swFace = swFace.GetNextFace
    Loop
    ReDim Preserve swFaceArr(UBound(swFaceArr) - 1)
    GetFacesWithAttribute = swFaceArr
End Function
```

```
Sub main()
```

```
    ' 1 = invisible
    ' 0 = visible
    Const CreateVisible As Long = 0

    Const sAttDefName As String = "TDE_temp_attrib"
    Const sAttRootName As String = "TDE"

    Dim swApp As SldWorks.SldWorks
    Dim swAttDef As SldWorks.AttributeDef
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swBody As SldWorks.Body2
    Dim swCopyBody As SldWorks.Body2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim nSelCount As Long
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swAtt() As SldWorks.Attribute
    Dim vFaceArr As Variant
    Dim swNewPart As SldWorks.PartDoc
    Dim swNewModel As SldWorks.ModelDoc2
    Dim swFeat As SldWorks.Feature
    Dim vBodies As Variant
    Dim i As Long
    Dim bLocChk As Boolean
    Dim nRetVal As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swAttDef = swApp.DefineAttribute(sAttDefName)
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swPart = swModel

    bRet = swAttDef.Register

    ' Add attribute to selected faces
    nSelCount = swSelMgr.GetSelectedObjectCount
    ReDim swAtt(nSelCount)
    For i = 1 To nSelCount
        Set swFace = swSelMgr.GetSelectedObject6(i, -1)
        Set swEnt = swFace
        Set swAtt(i - 1) = swAttDef.CreateInstance5(swModel, swEnt, sAttRootName & i, CreateVisible, swAllConfiguration)
    Next i

    vBodies = swPart.GetBodies2(swAllBodies, True)
    Set swBody = vBodies(0)
    Set swCopyBody = swBody.Copy
```



```

' Remove attribute from faces
For i = 1 To nSelCount
    bRet = swAtt(i - 1).Delete(True)
Next i

vFaceArr = GetFacesWithAttribute(swApp, swCopyBody, swAttDef)

' Can only delete faces from a temporary body
Debug.Assert swCopyBody.IsTemporaryBody

' Should not assert because it may fail to delete
' faces or fail local check or both
bRet = swCopyBody.DeleteFaces3(vFaceArr, 2, False, bLocChk)

' Should not assert because body may be a surface body
nRetVal = swCopyBody.Check2

Set swNewPart = swApp.NewPart
Set swNewModel = swNewPart
Set swFeat = swNewPart.CreateFeatureFromBody3(swCopyBody, False, swCreateFeatureBodyCheck)

End Sub

```

## Delete Instances and Dimensions in Variable Pattern Feature Example (C#)

This example shows how to:

- delete instances in a variable pattern feature.
- add and delete dimensions in the pattern table of a variable pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the part exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part and selects the variable pattern feature.
// 2. Accesses the variable pattern feature.
// 3. Deletes the last and fourth instance from the pattern table
//    and pattern.
// 4. Gets the number and names of the controlling dimensions.
// 5. Selects and adds two dimensions to the pattern table and
//    deletes the second dimension from the pattern table.
// 6. Rolls forward the modified variable pattern feature.
// 7. Examine the graphics area and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            DimPatternFeatureData swDimensionDrivenTablePatternFeat = default(DimPatternFeatureData);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int dimNbr = 0;
            int i = 0;
            string[] controllingDimNames = null;
            string controllingDimName = null;

            //Open part and select the variable pattern feature
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\bottle.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            status = swModelDocExt.SelectByID2("VarPattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swDimensionDrivenTablePatternFeat = (DimPatternFeatureData)swFeature.GetDefinition();

            //Roll back and access the variable pattern feature
            status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, null);

            //Delete the last instance of the variable pattern in the pattern table
            status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(-1);
            Debug.Print("Last instance of variable pattern deleted from pattern table? " + status);

            //Delete another instance of the variable pattern in the pattern table
            status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(3);
            Debug.Print("Fourth instance of variable pattern deleted from pattern table? " + status);

            //Get the number and names of the controlling dimensions
            dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount();
            Debug.Print("Number of controlling dimensions: " + dimNbr);
            Debug.Print("Controlling dimension names: ");
            Array.Resize(ref controllingDimNames, dimNbr + 1);
            for (i = 0; i <= dimNbr - 1; i++)
            {
                controllingDimNames[i] = (string)swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i);
                controllingDimName = (string)controllingDimNames[i];
                Debug.Print("    " + controllingDimName);
            }

            //Select two display dimensions and add them to the pattern table

            status = swModelDocExt.SelectByID2("D1@Sketch2@bottle.SLDPR1", "DIMENSION", 0.0176517231321772, 0.0149650532369545, -0.024323574651
3165, false, 0, null, 0);
            swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, -1);
            status = swDimensionDrivenTablePatternFeat.AddDimension();
            if (status)
            {
                Debug.Print("Added " + swDisplayDimension.GetNameForSelection() + " to pattern table");
            }

            swModel.ClearSelection2(true);

            status = swModelDocExt.SelectByID2("D2@Sketch2@bottle.SLDPR1", "DIMENSION", 0.019517663324148, 0.0297942417761803, -0.0237034236540
374, false, 0, null, 0);
```

```

swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, -1);
status = swDimensionDrivenTablePatternFeat.AddDimension();
if (status)
{
    Debug.Print("Added " + swDisplayDimension.GetNameForSelection() + " to pattern table");
}

//Delete the just-added display dimension from the pattern table
status = swDimensionDrivenTablePatternFeat.DeleteDimension(swDisplayDimension.GetNameForSelection());
if (status)
{
    Debug.Print(" Deleted " + swDisplayDimension.GetNameForSelection() + " from pattern table");
}

swDimensionDrivenTablePatternFeat.PropagateVisualProperties = true;

//Roll forward the variable pattern feature
swFeature.ModifyDefinition(swDimensionDrivenTablePatternFeat, swModel, null);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

## Delete Instances and Dimensions in Variable Pattern Feature Example (VBA)

This example shows how to:

- delete instances in a variable pattern feature.
- add and delete dimensions in the pattern table in a variable pattern table.

```

'-----
' Preconditions:
' 1. Verify that the part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part and selects the variable pattern feature.
' 2. Accesses the variable pattern feature.
' 3. Deletes the last and fourth instance from the pattern table
'    and pattern.
' 4. Gets the number and names of the controlling dimensions.
' 5. Selects and adds two dimensions to the pattern table and
'    deletes the second dimension from the pattern table.
' 6. Rolls forward the modified variable pattern feature.
' 7. Examine the graphics area and Immediate window.
'
' NOTE: Because the part used is elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swDimensionDrivenTablePatternFeat As SldWorks.DimPatternFeatureData
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim dimNbr As Long
Dim i As Long
Dim controllingDimNames() As String
Dim controllingDimName As String

Sub main()

    Set swApp = Application.SldWorks

    'Open part and select the variable pattern feature
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\bottle.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    status = swModelDocExt.SelectByID2("VarPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swDimensionDrivenTablePatternFeat = swFeature.GetDefinition

    'Roll back and access the variable pattern feature
    status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

    'Delete the last instance of the variable pattern in the pattern table and pattern
    status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(-1)
    Debug.Print "Last instance of variable pattern deleted from pattern table? " & status

    'Delete another instance of the variable pattern in the pattern table and pattern
    status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(3)
    Debug.Print "Fourth instance of variable pattern deleted from pattern table? " & status

    'Get the number and names of the controlling dimensions
    dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount
    Debug.Print ("Number of controlling dimensions: " & dimNbr)
    Debug.Print (" Controlling dimension names: ")
    ReDim Preserve controllingDimNames(dimNbr)
    For i = 0 To dimNbr - 1
        controllingDimNames(i) = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i)
        controllingDimName = controllingDimNames(i)
        Debug.Print (" " & controllingDimName)
    Next i

    'Select two display dimensions and add them to the pattern table
    status = swModelDocExt.SelectByID2("D1@Sketch2@bottle.SLDPRT", "DIMENSION", 1.76517231321772E-02, 1.49650532369545E-02, -2.43235746513165E-02, False, 0, Nothing, 0)
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
    status = swDimensionDrivenTablePatternFeat.AddDimension
    If status Then
        Debug.Print "Added " & swDisplayDimension.GetNameForSelection & " to pattern table"
    End If

    swModel.ClearSelection2 True

    status = swModelDocExt.SelectByID2("D2@Sketch2@bottle.SLDPRT", "DIMENSION", 0.019517663324148, 2.97942417761803E-02, -2.37034236540374E-02, False, 0, Nothing, 0)
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
    status = swDimensionDrivenTablePatternFeat.AddDimension
    If status Then
        Debug.Print "Added " & swDisplayDimension.GetNameForSelection & " to pattern table"
    End If

```

```
'Delete the just-added display dimension from the pattern table
status = swDimensionDrivenTablePatternFeat.DeleteDimension(swDisplayDimension.GetNameForSelection)
If status Then
    Debug.Print " Deleted " & swDisplayDimension.GetNameForSelection & " from pattern table"
End If

swDimensionDrivenTablePatternFeat.PropagateVisualProperties = True

'Roll forward the variable pattern feature
swFeature.ModifyDefinition swDimensionDrivenTablePatternFeat, swModel, Nothing

End Sub
```

## Delete Instances and Dimensions in Variable Pattern Feature Example (VB.NET)

This example shows how to:

- delete instances in a variable pattern feature.
- add and delete dimensions in the pattern table of a variable pattern feature.

```

-----
Preconditions:
1. Verify that the part exists.
2. Open the Immediate window.
Postconditions:
1. Opens the part and selects the variable pattern feature.
2. Accesses the variable pattern feature.
3. Deletes the last and fourth instance from the pattern table
   and pattern.
4. Gets the number and names of the controlling dimensions.
5. Selects and adds two dimensions to the pattern table and
   deletes the second dimension from the pattern table.
6. Rolls forward the modified variable pattern feature.
7. Examine the graphics area and Immediate window.
NOTE: Because the part used is elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swDimensionDrivenTablePatternFeat As DimPatternFeatureData
        Dim swDisplayDimension As DisplayDimension
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim dimNbr As Integer
        Dim i As Integer
        Dim controllingDimNames() As String
        Dim controllingDimName As String

        'Open part and select the variable pattern feature
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\bottle.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        status = swModelDocExt.SelectByID2("VarPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swDimensionDrivenTablePatternFeat = swFeature.GetDefinition

        'Roll back and access the variable pattern feature
        status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

        'Delete the last instance of the variable pattern in the pattern table and pattern
        status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(-1)
        Debug.Print("Last instance of variable pattern deleted from pattern table?" & status)

        'Delete another instance of the variable pattern in the pattern table and pattern
        status = swDimensionDrivenTablePatternFeat.DeleteInstanceAt(3)
        Debug.Print("Fourth instance of variable pattern deleted from pattern table?" & status)

        'Get the number and names of the controlling dimensions
        dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount
        Debug.Print("Number of controlling dimensions: " & dimNbr)
        Debug.Print("Controlling dimension names: ")
        ReDim Preserve controllingDimNames(dimNbr)
        For i = 0 To dimNbr - 1
            controllingDimNames(i) = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i)
            controllingDimName = controllingDimNames(i)
            Debug.Print("    " & controllingDimName)
        Next i

        'Select two display dimensions and add them to the pattern table

        status = swModelDocExt.SelectByID2("D1@Sketch2@bottle.SLDPRT", "DIMENSION", 0.0176517231321772, 0.0149650532369545, -0.0243235746513165, False, 0, Nothing, 0)
        swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
        status = swDimensionDrivenTablePatternFeat.AddDimension
        If status Then
            Debug.Print("Added " & swDisplayDimension.GetNameForSelection & " to pattern table")
        End If

        swModel.ClearSelection2(True)

        status = swModelDocExt.SelectByID2("D2@Sketch2@bottle.SLDPRT", "DIMENSION", 0.019517663324148, 0.0297942417761803, -0.0237034236540374, False, 0, Nothing, 0)
        swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
        status = swDimensionDrivenTablePatternFeat.AddDimension
        If status Then
            Debug.Print("Added " & swDisplayDimension.GetNameForSelection & " to pattern table")
        End If

        'Delete the just-added display dimension from the pattern table
        status = swDimensionDrivenTablePatternFeat.DeleteDimension(swDisplayDimension.GetNameForSelection)
        If status Then
            Debug.Print("Deleted " & swDisplayDimension.GetNameForSelection & " from pattern table")
        End If
    End Sub
End Class

```

```
swDimensionDrivenTablePatternFeat.PropagateVisualProperties = True  
  
'Roll forward the variable pattern feature  
swFeature.ModifyDefinition(swDimensionDrivenTablePatternFeat, swModel, Nothing)
```

```
End Sub
```

```
''' <summary>  
''' The Sldworks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

## Delete OLE Objects from Drawing Sheet and Template Example (VBA)

This example shows how to delete any OLE objects from the current drawing sheet and template.

```

'-----
' Preconditions:
' 1. Open a drawing that contains one or more
'    OLE objects in the drawing sheet or template.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Deletes all OLE objects in drawing sheet
'    and template.
' 2. Examine the graphics area and Immediate window.
'-----

Option Explicit

Sub DeleteAllOleItemsInSheet _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swSheet As SldWorks.Sheet _
)
    Dim nOleCnt          As Long
    Dim i                As Long
    Dim nSize            As Long
    Dim nAspect          As Long
    Dim skArray()        As Variant
    Dim bRet             As Boolean

    nOleCnt = swSheet.GetOLEObjectCount: If 0 = nOleCnt Then Exit Sub
    ReDim skArray(nOleCnt - 1)

    ' Get location of each OLE item
    For i = 0 To nOleCnt - 1
        skArray(i) = swSheet.GetOLEObjectSettings(i, nSize, nAspect)
    Next i

    ' Delete each OLE items
    For i = 0 To nOleCnt - 1
        bRet = swModel.SelectByID("", "OLEITEM", 0.5 * (skArray(i)(0) + skArray(i)(3)), 0.5 * (skArray(i)(1) + skArray(i)(4)), 0.5 * (skArray(i)(2) + skArray(i)(5)))
        bRet = swModel.DeleteSelection(False)
        Debug.Print ("OLE item " & i & " deleted?" & bRet)
    Next i
End Sub

Sub main()
    Dim swApp            As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swDraw           As SldWorks.DrawingDoc
    Dim swSheet          As Object

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSheet = swDraw.GetCurrentSheet

    DeleteAllOleItemsInSheet swApp, swModel, swSheet

    ' Switch to template and delete OLE objects
    swDraw.EditTemplate
    Set swSheet = swDraw.GetCurrentSheet
    DeleteAllOleItemsInSheet swApp, swModel, swSheet

    swDraw.EditSheet

    ' Switch back to sheet and make sure all OLE objects are deleted
    Set swSheet = swDraw.GetCurrentSheet
End Sub

```



### Delete Selected Faces Example (VBA)

This example shows how to delete selected faces.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\block.sldprt.  
' 2. Select the cylindrical faces.  
'  
' Postconditions:  
' 1. Deletes the cylindrical faces, but does not fill  
'    in the deleted cylindrical faces. Also, creates a  
'    a DeleteFace feature.  
' 2. Examine the graphics area and FeatureManager  
'    design tree.  
'  
' NOTE: Because this part is used elsewhere, do not  
' save changes.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    ' IModelDoc2::InsertDeleteFace2 only works for parts  
    ' 1 refills the face after it is deleted  
    ' 0 does not  
    bRet = swModel.InsertDeleteFace2(0)
```

End Sub

### Delete Selected Feature Example (VBA)

This example shows how to delete a selected feature. You can specify to delete the absorbed or children features or both types of features. This example does not ask you to confirm the deletion.

```
'-----
' Preconditions:
' 1. Open a model document.
' 2. Select the feature to delete.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Deletes the selected feature.
' 2. Examine the Immediate window, graphics
'    area, and FeatureManager design tree.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim DeleteOption As Long
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    ' To delete absorbed features, use enum swDeleteSelectionOptions_e.swDelete_Absorbed
    ' To delete children features, use enum swDeleteSelectionOptions_e.swDelete_Children
    ' To keep absorbed features and children features, set DeleteOption = 0
    DeleteOption = swDeleteSelectionOptions_e.swDelete_Absorbed

    'Comment out the previous statement and uncomment one of the
    'following statements to change how to delete the selected feature
    'DeleteOption = swDeleteSelectionOptions_e.swDelete_Children
    'DeleteOption = 0
    'DeleteOption =swDeleteSelectionOptions_e.swDelete_Absorbed + swDeleteSelectionOptions_e.swDelete_Children

    status = swModelDocExt.DeleteSelection2(DeleteOption)
    Debug.Print "Feature deleted? " & status

End Sub
```

## Delete a Smart Feature Example (C#)

This example shows how to delete a feature from a Smart Component.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\holecube.sldprt.
// 2. Expand the Smart Feature and Features folders to verify that
//    Features contains two extrusion features.
//
// Postconditions:
// 1. Deletes one extrusion feature from the Features folder.
// 2. Expand the Smart Feature and Features folders to verify that
//    Features contains one extrusion feature.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
```

```
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace DeleteSmartFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        FeatureManager featMgr;
        Feature myFeature;
        ModelDoc2 swTrainAss;
        ModelDoc2 swSmartPart;
        SmartComponentFeatureData swSCFD;
        SelectionMgr selMgr;
        Feature feat;
        object[]
feats;
        object obj;
        int i;
        int count;
        bool boolstatus;

        public void Main()
        {
            swSmartPart = (ModelDoc2)swApp.ActiveDoc;
            featMgr = swSmartPart.FeatureManager;
            feats = (object[])featMgr.GetFeatures(true);
            count = featMgr.GetFeatureCount(true);

            for (i = 0; i <= count - 1; i++)
            {
                myFeature = (Feature)feats[i];
                if (myFeature.Name == "Smart Feature")
                {
                    Debug.Print(myFeature.Name);
                }
            }
        }
    }
}
```

```

        swSCFD = (SmartComponentFeatureData)myFeature.GetDefinition();

        // Open the training
        assembly of the Smart Component

        //
        Open the SmartComponent PropertyManager page to access its selection lists
        boolstatus = swSCFD.AccessSelections(true);

        swTrainAss = (ModelDoc2)swApp.ActiveDoc;
        selMgr = (SelectionMgr)swTrainAss.SelectionManager;

        Debug.Print("Number of features: " +
selMgr.GetSelectedObjectCount2((int)swSmartComponentSelectionTypes_e.swSmartComponentFeatures));

        Debug.Print("Number of components: " +
selMgr.GetSelectedObjectCount2((int)swSmartComponentSelectionTypes_e.swSmartComponentComponents));

        // Get the first
        extrusion from the features selection list
        obj = selMgr.GetSelectedObject6(1,
(int)swSmartComponentSelectionTypes_e.swSmartComponentFeatures);
        feat = (Feature)obj;

        // To delete an
        extrusion feature from a Smart Component,

        //
        re-select it in the Smart Features selection list
        boolstatus = feat.Select2(true,
(int)swSmartComponentSelectionTypes_e.swSmartComponentFeatures);

        // Modify the
        definition of the Smart Feature,

        //
        close the training assembly, and rebuild the Smart Component
        boolstatus = myFeature.ModifyDefinition(swSCFD, swSmartPart, null);
    }
}

}

public SldWorks swApp;

}
}

```

## Delete a Smart Feature Example (VBA)

This example shows how to delete a feature from a Smart Component.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\holecube.sldprt.
' 2. Expand the Smart Feature and Features folders to verify that
'    Features contains two extrusion features.
'
' Postconditions:
' 1. Deletes one extrusion feature from the Features folder.
' 2. Expand the Smart Feature and Features folders to verify that
'    Features contains one extrusion feature.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim featMgr As SldWorks.FeatureManager
Dim myFeature As SldWorks.Feature
Dim swTrainAss As SldWorks.ModelDoc2
Dim swSmartPart As SldWorks.ModelDoc2
Dim swSCFD As SldWorks.SmartComponentFeatureData
Dim selMgr As SldWorks.SelectionMgr
Dim feat As SldWorks.Feature
Dim feats As Variant
Dim obj As Object
Dim i As Long
Dim count As Long
Dim boolstatus As Boolean

Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set swSmartPart = swApp.ActiveDoc
    Set featMgr = swSmartPart.FeatureManager
    feats = featMgr.GetFeatures(True)
    count = featMgr.GetFeatureCount(True)

    For i = 0 To count - 1
        Set myFeature = feats(i)
        If myFeature.Name = "Smart Feature" Then
            Set swSCFD = myFeature.GetDefinition

            ' Open the training assembly of the Smart Component
            ' Open the SmartComponent PropertyManager page to access its selection lists
            boolstatus = swSCFD.AccessSelections(True):

            Set swTrainAss = swApp.ActiveDoc
            Set selMgr = swTrainAss.SelectionManager

            Debug.Print "Number of features: " & selMgr.GetSelectedObjectCount2(swSmartComponentFeatures)
            Debug.Print "Number of components: " & selMgr.GetSelectedObjectCount2(swSmartComponentComponents)

            ' Get the first extrusion from the features selection list
            Set obj = selMgr.GetSelectedObject6(1, swSmartComponentFeatures)
            Set feat = obj

            ' To delete an extrusion feature from a Smart Component,
            ' re-select it in the Smart Features selection list
            boolstatus = feat.Select2(True, swSmartComponentFeatures)

            ' Modify the definition of the Smart Feature,
            ' close the training assembly, and rebuild the Smart Component
            boolstatus = myFeature.ModifyDefinition(swSCFD, swSmartPart, Nothing)
        End If
    Next

End Sub

```

## Delete a Smart Feature Example (VB.NET)

This example shows how to delete a feature from a Smart Component.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\holecube.sldprt.
' 2. Expand the Smart Feature and Features folders to verify that
'    Features contains two extrusion features.
'
' Postconditions:
' 1. Deletes one extrusion feature from the Features folder.
' 2. Expand the Smart Feature and Features folders to verify that
'    Features contains one extrusion feature.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim featMgr As FeatureManager
    Dim myFeature As Feature
    Dim swTrainAss As ModelDoc2
    Dim swSmartPart As ModelDoc2
    Dim swSCFD As SmartComponentFeatureData
    Dim selMgr As SelectionMgr
    Dim feat As Feature
    Dim feats As Object
    Dim obj As Object
    Dim i As Long
    Dim count As Long
    Dim boolstatus As Boolean

    Sub main()

        swSmartPart = swApp.ActiveDoc
        featMgr = swSmartPart.FeatureManager
        feats = featMgr.GetFeatures(True)
        count = featMgr.GetFeatureCount(True)

        For i = 0 To count - 1
            myFeature = feats(i)

            If myFeature.Name = "Smart Feature" Then
                Debug.Print(myFeature.Name)

                swSCFD = myFeature.GetDefinition

                ' Open the training
                assembly of the Smart Component
                ,
                Open the SmartComponent PropertyManager page to access its selection lists

                boolstatus = swSCFD.AccessSelections(True)

                swTrainAss = swApp.ActiveDoc
                selMgr = swTrainAss.SelectionManager

                Debug.Print("Number of

```

```

features: " & selMgr.GetSelectedObjectCount2(swSmartComponentSelectionTypes_e.swSmartComponentFeatures))
    Debug.Print("Number of
components: " & selMgr.GetSelectedObjectCount2(swSmartComponentSelectionTypes_e.swSmartComponentComponents))

    ' Get the first extrusion
from the features selection list
    obj = selMgr.GetSelectedObject6(1, swSmartComponentSelectionTypes_e.swSmartComponentFeatures)
    feat = obj

    ' To delete an extrusion
feature from a Smart Component,
    '
re-select it in the Smart Features selection list
    boolstatus = feat.Select2(True, swSmartComponentSelectionTypes_e.swSmartComponentFeatures)

    ' Modify the definition of
the Smart Feature,
    '
close the training assembly, and rebuild the Smart Component
    boolstatus = myFeature.ModifyDefinition(swSCFD, swSmartPart, Nothing)

End If

Next

End Sub

Public swApp As SldWorks

End Class

```

## Delete Subassemblies Example (C#)

This example shows how to delete the subassembly in which the selected subassembly is a component.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects and deletes the [Assem4^Assem3_assemSubAssems]<1>
// subassembly and the subassembly in which it is a component.
// 3. Examine the FeatureManager design tree and Immediate
// window.
//
// NOTE: Because the assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1.CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\assemSubAssems.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
rrors, ref warnings);
            swAssembly = (AssemblyDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Assem3^assemSubAssems-1@assemSubAssems/Assem4^Assem3_assemSubAssems-
1@Assem3^assemSubAssems", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swAssembly.DeleteSelections((int)swAssemblyDeleteOptions_e.swDelete_SubAssembly);
            Debug.Print("Selected subassembly and the subassembly in which it is a component deleted? " + status);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```



### Delete Subassemblies Example (VBA)

This example shows how to delete the subassembly in which the selected subassembly is a component.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Selects and deletes the [Assem4^Assem3_assemSubAssems]<1>
'    subassembly and the subassembly in which it is a component.
' 3. Examine the FeatureManager design tree and Immediate
'    window.
'
' NOTE: Because the assembly is used elsewhere, do not save
'       changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\assemSubAssems.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssembly = swModel
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Assem3^assemSubAssems-1@assemSubAssems/Assem4^Assem3_assemSubAssems-1@Assem3^assemSubAssems", "COMPONENT", 0, 0,
0, False, 0, Nothing, 0)
    status = swAssembly.DeleteSelections(swAssembly.DeleteOptions_e.swDelete_SubAssembly)
    Debug.Print "Selected subassembly and the subassembly in which it is a component deleted? " & status

End Sub
```

## Delete Subassemblies Example (VB.NET)

This example shows how to delete the subassembly in which the selected subassembly is a component.

```

-----
Preconditions:
1. Verify that the specified assembly exists.
2. Open the Immediate window.
Postconditions:
1. Opens the assembly.
2. Selects and deletes the [Assem4^Assem3_assemSubAssems]<1>
subassembly and the subassembly in which it is a component.
3. Examine the FeatureManager design tree and Immediate
window.
NOTE: Because the assembly is used elsewhere, do not save
changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssembly As AssemblyDoc
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\assemSubAssems.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swAssembly = swModel
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Assem3^assemSubAssems-1@assemSubAssems/Assem4^Assem3_assemSubAssems-
1@Assem3^assemSubAssems", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swAssembly.DeleteSelections(swAssemblyDeleteOptions_e.swDelete_SubAssembly)
        Debug.Print("Selected subassembly and the subassembly in which it is a component deleted? " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

## Deselect Every Other Selected Object Example (C++ COM)

This example shows how to deselect every other object.

```
// -----
void APITestFunction()
{
    HRESULT          hr = S_OK;
    long             bRet = FALSE;
    long             nRetVal = -1;
    CComPtr <ISldWorks> pSldWorks;
    CComPtr <IModelDoc2> pModel;
    CComPtr <ISelectionMgr> pSelMgr;
    long*            nDeselectArr = NULL;
    long             nDeselectCount = -1;
    long             nOldSelCount = -1;
    long             nNewSelCount = -1;
    long             i = -1;
    // Connect to SOLIDWORKS Design application
    pSldWorks = TheApplication->GetSWApp();
    ASSERT(pSldWorks);
    hr = pSldWorks->get_IActiveDoc2(&pModel);
    ASSERT(pModel);
    hr = pModel->get_ISelectionManager(&pSelMgr);
    ASSERT(pSelMgr);
    hr = pSelMgr->GetSelectedObjectCount(&nOldSelCount);
    ASSERT(nOldSelCount > 1);
    nDeselectCount = floor(nOldSelCount / 2);
    nDeselectArr = new long[nDeselectCount];
    ASSERT(nDeselectArr);
    ZeroMemory(nDeselectArr, nDeselectCount * sizeof(long));
    for (i = 0; i < nDeselectCount; i++)
    {
        // Deselect every other item
        // Memory array is 0-based, but SelectionMgr array is 1-based
        nDeselectArr[i] = 2 * i + 1;
    }
    hr = pSelMgr->IDeselect(nDeselectCount, nDeselectArr, &bRet);
    ASSERT(bRet);
    hr = pSelMgr->GetSelectedObjectCount(&nNewSelCount);
    ASSERT(nNewSelCount == nOldSelCount - nDeselectCount);
    delete [] nDeselectArr;
}
// -----
```

## Deselect Marked Entity Example (C#)

This example shows how to deselect a previously selected and marked entity.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects the Base-Extrude feature.
// 2. Examine the graphics area and FeatureManager
//    design tree to verify.
// 3. At System.Diagnostics.Debugger.Break(), press F5
//    to continue.
// 4. Deselects the Base-Extrude feature.
// 5. Examine the Immediate window, graphics area, and
//    FeatureManager design tree to verify.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DeSelect2ObjectCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DocumentSpecification swDocSpecification = default(DocumentSpecification);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelExt = default(ModelDocExtension);
            bool bRet = false;
            int lMark = 0;
            int lMarkedIdx = 0;
            int lNumMarkedSelections = 0;
            int lNumAllSelections = 0;

            // Open the document
            swDocSpecification = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\swutilities\\bracket_a.sldprt");
            swModel = (ModelDoc2)swApp.ActiveDoc;
            if (swModel == null)
            {
                swModel = (ModelDoc2)swApp.OpenDoc7(swDocSpecification);
            }

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelExt = (ModelDocExtension)swModel.Extension;
            // Select the Base-Extrude feature and mark it with a value of 8
            bRet = swModelExt.SelectByID("Base-Extrude", "BODYFEATURE", 0, 0, 0, false, 8, null, (int)swSelectOption_e.swSelectOptionDefault);

            System.Diagnostics.Debugger.Break();
            // Examine the graphics area to verify
            // that Base-Extrude is selected
            // Press F5 to continue

            // Look for a given mark value
            lMark = 8;
            // Get the number of marked selections
            lNumMarkedSelections = swSelMgr.GetSelectedObjectCount2(lMark);
            Debug.Print("Number of marked selections = " + lNumMarkedSelections);

            // Get the total number of selections
            lNumAllSelections = swSelMgr.GetSelectedObjectCount2(-1);
            Debug.Print("Number of selections = " + lNumAllSelections);

            Debug.Print(" ");

            // Deselect the marked selection
            for (lMarkedIdx = 1; lMarkedIdx <= lNumAllSelections; lMarkedIdx++)
            {
                Debug.Print("Selected object #" + lMarkedIdx + " deselected? " + swSelMgr.DeSelect2(lMarkedIdx, lMark));
            }

            // Examine the graphics area to verify that Base-Extrude is deselected
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

## Deselect Marked Entity Example (VBA)

This example shows how to deselect a previously selected and marked entity.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects the Base-Extrude feature.
' 2. Examine the graphics area and FeatureManager
'    design tree to verify.
' 3. At Stop, press F5 to continue.
' 4. Deselects the Base-Extrude feature.
' 5. Examine the Immediate window, graphics
'    area, and FeatureManager design tree to verify.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDocSpecification As SldWorks.DocumentSpecification
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelExt As SldWorks.ModelDocExtension
Dim bRet As Boolean
Dim lMark As Long
Dim lMarkedIdx As Long
Dim lNumMarkedSelections As Long
Dim lNumAllSelections As Long

Sub main()

Set swApp = Application.SldWorks

' Open the document
Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\swutilities\bracket_a.sldprt")
Set swModel = swApp.ActiveDoc
If swModel Is Nothing Then
Set swModel = swApp.OpenDoc7(swDocSpecification)
End If

Set swSelMgr = swModel.SelectionManager
Set swModelExt = swModel.Extension
' Select the Base-Extrude feature and mark with a value of 8
bRet = swModelExt.SelectByID2("Base-Extrude", "BODYFEATURE", 0, 0, 0, False, 8, Nothing, swSelectOptionDefault)

Stop
' Examine the part to verify that Base-Extrude was selected
' Press F5 to continue

' Look for a given mark value
lMark = 8
' Get the number of marked selections
lNumMarkedSelections = swSelMgr.GetSelectedObjectCount2(lMark)
Debug.Print "Number of marked selections = " & lNumMarkedSelections

' Get the total number of selections
lNumAllSelections = swSelMgr.GetSelectedObjectCount2(-1)
Debug.Print "Number of selections = " & lNumAllSelections

Debug.Print " "

' Deselect the marked selection
For lMarkedIdx = 1 To lNumAllSelections
Debug.Print ("Selected object #" & lMarkedIdx & " deselected? " & swSelMgr.DeSelect2(lMarkedIdx, lMark))
Next lMarkedIdx

' Examine the graphics area to verify that Base-Extrude is deselected

End Sub

```

## Deselect Marked Entity (VB.NET)

This example shows how to deselect a previously selected and marked entity.

```

-----
Preconditions:
1. Verify that the part to open exists.
2. Open the Immediate window.
-----
Postconditions:
1. Selects the Base-Extrude feature.
2. Examine the graphics area and FeatureManager design tree to verify.
3. At Stop, press F5 to continue.
4. Deselects the Base-Extrude feature.
5. Examine the Immediate window, graphics area, and FeatureManager design tree to verify.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDocSpecification As DocumentSpecification
        Dim swSelMgr As SelectionMgr
        Dim swModelExt As ModelDocExtension
        Dim bRet As Boolean
        Dim lMark As Integer
        Dim lMarkedIdx As Integer
        Dim lNumMarkedSelections As Integer
        Dim lNumAllSelections As Integer

        ' Open the document
        swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\swutilities\bracket_a.sldprt")
        swModel = swApp.ActiveDoc
        If swModel Is Nothing Then
            swModel = swApp.OpenDoc7(swDocSpecification)
        End If

        swSelMgr = swModel.SelectionManager
        swModelExt = swModel.Extension
        ' Select the Base-Extrude feature and mark it with a a value of 8
        bRet = swModelExt.SelectByID2("Base-Extrude", "BODYFEATURE", 0, 0, 0, False, 8, Nothing, swSelectOption_e.swSelectOptionDefault)

        Stop
        ' Examine the graphics area to verify that Base-Extrude is selected
        ' Press F5 to continue

        ' Look for a given mark value
        lMark = 8
        ' Get the number of marked selections
        lNumMarkedSelections = swSelMgr.GetSelectedObjectCount2(lMark)
        Debug.Print("Number of marked selections = " & lNumMarkedSelections)

        ' Get the total number of selections
        lNumAllSelections = swSelMgr.GetSelectedObjectCount2(-1)
        Debug.Print("Number of selections = " & lNumAllSelections)
        Debug.Print(" ")

        ' Deselect the marked selection
        For lMarkedIdx = 1 To lNumAllSelections
            Debug.Print("Selected object #" & lMarkedIdx & " deselected? " & swSelMgr.DeSelect2(lMarkedIdx, lMark))
        Next lMarkedIdx

        ' Examine the graphics area to verify that Base-Extrude is deselected

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

## Detect Interferences Using a Transform Example (C#)

This example shows how to specify a transform to detect interferences.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\InterferenceAssem.sldasm.
// 2. Uncomment one of the userOpt assignments.
//
// Postconditions:
// 1. When the macro stops, observe the position of the components in the
//    graphics area.
// 2. Press F5 to continue.
// 3. Inspect the Immediate Window for interfering components.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace IntDet_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swDoc;
        AssemblyDoc swADoc;
        SelectionMgr swSM;
        InterferenceDetectionMgr swIDM;
        Component2[] swComp;
        Component2 comp;
        Object[] varComp;
        int numOfComp;
        MathTransform[] swCompTrans;
        Object[] varCompTrans;
        bool isInterfering;
        object outcomp;
        Object[] outIntComp;
        double gposX;
        double gposY;
        int I;

        // Uncomment one of the following lines to transform the components
        //int userOpt = 0; //make components coincident
        int userOpt = 1; //make components intersecting

        public void Main()
        {
            swDoc = (ModelDoc2)swApp.ActiveDoc;
            swADoc = (AssemblyDoc)swDoc;
            swSM = (SelectionMgr)swDoc.SelectionManager;
            SetPosAsPerOption();
            SelectAllComponent();
            isInterfering = false;
            numOfComp = swSM.GetSelectedObjectCount();

            if ((numOfComp > 0))
            {
                getSelectedComp();
                PrintComponentName(varComp);
                CreateSameTransMatAsCompCount();
                ChangePosOfCompAsPerOption();

                varCompTrans = swCompTrans;

                swIDM = swADoc.InterferenceDetectionManager;
                swIDM.TreatCoincidenceAsInterference = true;
                isInterfering = swIDM.GetComponentsTransformInterference((varComp), (varCompTrans), out (outcomp));

                outIntComp = (Object[])outcomp;

                // Observe transformation of objects for interference detection
                System.Diagnostics.Debugger.Break();

                swIDM.Done(); // Interference detection finished

                PrintInterferenceInfo();
                varComp = null;
            }
            else
            {
                System.Windows.Forms.MessageBox.Show("Select components");
            }
        }
    }
}
```

```

public void SetPosAsPerOption()
{
    if (userOpt == 0)
    {
        gposX = 100.0 / 1000.0;
        gposY = -50.0 / 1000.0;
    }
    else
    {
        gposX = 70.0 / 1000.0;
        gposY = -50.0 / 1000.0;
    }
}

public void SelectAllComponent()
{
    bool boolstatus = false;
    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Red-1@InterferenceAssem", "COMPONENT", 0, 0, 0, false, 0, null, 0);
    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Green-1@InterferenceAssem", "COMPONENT", 0, 0, 0, true, 0, null,
0);

    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Blue-1@InterferenceAssem", "COMPONENT", 0, 0, 0, true, 0, null, 0);
    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Orange-1@InterferenceAssem", "COMPONENT", 0, 0, 0, true, 0, null,
0);

    boolstatus = swDoc.Extension.SelectByID2("SquarePad_SkyBlue-1@InterferenceAssem", "COMPONENT", 0, 0, 0, true, 0, null,
0);

    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Yellow-1@InterferenceAssem", "COMPONENT", 0, 0, 0, true, 0, null,
0);

    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Pink-1@InterferenceAssem", "COMPONENT", 0, 0, 0, true, 0, null, 0);
}

public void PrintInterferenceInfo()
{
    if ((isInterfering))
    {
        Debug.Print("Interfering components:");
        Debug.Print("");
        for (I = 0; I <= outIntComp.GetUpperBound(0); I++)
        {
            Component2 swOutComp = default(Component2);
            swOutComp = (Component2)outIntComp[I];
            Debug.Print(swOutComp.Name2);
            swOutComp.Select4(true, null, false);
            swOutComp = null;
        }
    }
    else
    {
        Debug.Print("No interference");
    }
}

public void getSelectedComp()
{
    swComp = new Component2[numOfComp];
    for (I = 1; I <= numOfComp; I++)
    {
        swComp[I-1] = (Component2)swSM.GetSelectedObject6(I, -1);
    }
    varComp = swComp;
}

public void PrintComponentName(Object[] varComp)
{
    if (!((varComp == null)))
    {
        if (userOpt == 0)
        {
            Debug.Print("Detect interferences between coincident components:");
            Debug.Print("");
        }
        else
        {
            Debug.Print("Detect interferences between intersecting components:");
            Debug.Print("");
        }

        for (I = 0; I <= varComp.GetUpperBound(0); I++)
        {
            comp = default(Component2);
            comp = (Component2)varComp[I];

            Debug.Print("Component name : " + comp.Name2);
            comp = null;
        }
        Debug.Print("");
    }
}

public void CreateSameTransMatAsCompCount()
{
    swCompTrans = new MathTransform[numOfComp];
    if (!((varComp == null)))
    {
        for (I = 0; I <= varComp.GetUpperBound(0); I++)
        {
            Component2 comp = default(Component2);
            comp = (Component2)varComp[I];
            swCompTrans[I] = comp.Transform2;
            comp = null;
        }
    }
}

```



```

    }
}
public void ChangePosOfCompAsPerOption()
{
    ChangeRefPosOfComp(gposX, gposY);
}

public void ChangeRefPosOfComp(double posX, double posY)
{
    if (!(varComp == null))
    {
        for (I = 0; I <= varComp.GetUpperBound(0); I++)
        {
            if ((I > 0))
            {
                double[] varData = null;
                varData = (double[])swCompTrans[I].ArrayData;
                varData[9] = posX * I;
                varData[10] = posY * I;
                swCompTrans[I].ArrayData = varData;
            }
        }
    }
}

public SldWorks swApp;
}
}

```

## Detect Interferences Using a Transform Example (VBA)

This example shows how to specify a transform to detect interferences.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\InterferenceAssem.sldasm.
' 2. Uncomment one of the userOpt assignments.
'
' Postconditions:
' 1. When the macro stops, observe the position of the components in the
'    graphics area.
' 2. Press F5 to continue.
' 3. Inspect the Immediate Window for interfering components.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim swADoc As SldWorks.AssemblyDoc
Dim swSM As SldWorks.SelectionMgr
Dim swIDM As InterferenceDetectionMgr
Dim swComp() As SldWorks.Component2
Dim varComp As Variant
Dim numOfComp As Long
Dim swCompTrans() As SldWorks.MathTransform
Dim varCompTrans As Variant
Dim isInterfering As Boolean
Dim outIntComp As Variant
Dim outIntFaces As Variant
Dim gposX As Double
Dim gposY As Double
Dim I As Long

Option Explicit

' Uncomment one of the following lines to transform the components
' Const userOpt = 0 'make components coincident
' Const userOpt = 1 'make components intersecting

Sub main()

    Set swApp = Application.SldWorks
    Set swDoc = swApp.ActiveDoc
    Set swADoc = swDoc
    Set swSM = swDoc.SelectionManager
    SetPosAsPerOption
    SelectAllComponent
    isInterfering = False
    numOfComp = swSM.GetSelectedObjectCount
    If (numOfComp) Then

        getSelectedComp
        PrintComponentName varComp
        CreateSameTransMatAsCompCount
        ChangePosOfCompAsPerOption

        varCompTrans = swCompTrans

        Set swIDM = swADoc.InterferenceDetectionManager
        swIDM.TreatCoincidenceAsInterference = True
        isInterfering = swIDM.GetComponentsTransformInterference((varComp), (varCompTrans), outIntComp)

        Stop ' Observe transformation of objects for interference detection

        swIDM.Done ' Interference detection stopped
        PrintInterferenceInfo
        varComp = Empty
    Else
        MsgBox ("Select components")
    End If
End Sub

Sub SetPosAsPerOption()
    If (userOpt = 0) Then
        gposX = 100 / 1000
        gposY = -50 / 1000
    ElseIf (userOpt = 1) Then
        gposX = 70 / 1000
        gposY = -50 / 1000
    End If
End Sub

Sub SelectAllComponent()
    Dim boolstatus As Boolean
    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Red-1@InterferenceAssem", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = swDoc.Extension.SelectByID2("SquarePad_Green-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)

```

```

boolstatus = swDoc.Extension.SelectByID2("SquarePad_Blue-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Orange-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_SkyBlue-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Yellow-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Pink-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
End Sub
Sub PrintInterferenceInfo()
    If (isInterfering) Then
        Debug.Print ("Interfering components:")
        Debug.Print ("")
        For I = 0 To UBound(outIntComp)
            Dim swOutComp As SldWorks.Component2
            Set swOutComp = outIntComp(I)
            Debug.Print (swOutComp.Name2)
            swOutComp.Select4 True, Nothing, False
            Set swOutComp = Nothing
        Next I
    Else
        Debug.Print ("No interference")
    End If
End Sub
Sub getSelectedComp()
    ReDim swComp(numOfComp - 1)
    For I = 1 To numOfComp
        Set swComp(I - 1) = swSM.GetSelectedObject6(I, -1)
    Next I
    varComp = swComp
End Sub
Sub PrintComponentName(varComp As Variant)
    If Not (IsEmpty(varComp)) Then
        If userOpt = 0 Then
            Debug.Print ("Detect interferences between coincident components:")
            Debug.Print ("")
        Else
            Debug.Print ("Detect interferences between intersecting components:")
            Debug.Print ("")
        End If
    End If

    For I = LBound(varComp) To UBound(varComp)
        Dim swComp As SldWorks.Component2
        Set swComp = varComp(I)

        Debug.Print ("Component name : " & swComp.Name2)
        Set swComp = Nothing
    Next I
End Sub
Debug.Print ("")
End Sub
Sub CreateSameTransMatAsCompCount()
    ReDim swCompTrans(numOfComp - 1)
    If Not (IsEmpty(varComp)) Then
        For I = LBound(varComp) To UBound(varComp)
            Dim swComp As SldWorks.Component2
            Set swComp = varComp(I)
            Set swCompTrans(I) = swComp.Transform2
            Set swComp = Nothing
        Next I
    End If
End Sub
Sub ChangePosOfCompAsPerOption()
    ChangeRefPosOfComp gposX, gposY
End Sub
Sub ChangePosOfComp(compNum As Long, pos As Double)
    Dim varData As Variant
    varData = swCompTrans(compNum).ArrayData
    varData(9) = pos
    swCompTrans(compNum).ArrayData = varData
End Sub
Sub ChangeRefPosOfComp(posX As Double, posY As Double)
    If Not (IsEmpty(varComp)) Then
        For I = LBound(varComp) To UBound(varComp)
            If (I > 0) Then
                Dim varData As Variant
                varData = swCompTrans(I).ArrayData
                varData(9) = posX * I
                varData(10) = posY * I
                swCompTrans(I).ArrayData = varData
                If (userOpt = 2) Then
                    swComp(I).Transform2 = swCompTrans(I)
                    swDoc.ForceRebuild3 True
                End If
            End If
        Next I
    End If
End Sub

```

**Detect Interferences Using a Transform Example (VB.NET)**

This example shows how to specify a transform to detect interferences.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\InterferenceAssem.sldasm.
' 2. Uncomment one of the userOpt assignments.
'
' Postconditions:
' 1. When the macro stops, observe the position of the components in the
'    graphics area.
' 2. Press F5 to continue.
' 3. Inspect the Immediate Window for interfering components.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swDoc As ModelDoc2
    Dim swADoc As AssemblyDoc
    Dim swSM As SelectionMgr
    Dim swIDM As InterferenceDetectionMgr
    Dim swComp() As Component2
    Dim varComp As Object
    Dim numOfComp As Long
    Dim swCompTrans() As MathTransform
    Dim varCompTrans As Object
    Dim isInterfering As Boolean
    Dim outIntComp As Object
    Dim outIntFaces As Object
    Dim gposX As Double
    Dim gposY As Double
    Dim I As Long

    ' Uncomment one of the following lines to transform the components
    'Dim userOpt As Long = 0 'make components coincident
    Dim userOpt As Long = 1 'make components intersecting

    Sub main()

        swDoc = swApp.ActiveDoc
        swADoc = swDoc
        swSM = swDoc.SelectionManager
        SetPosAsPerOption()
        SelectAllComponent()
        isInterfering = False
        numOfComp = swSM.GetSelectedObjectCount
        If (numOfComp) Then

            getSelectedComp()
            PrintComponentName(varComp)
            CreateSameTransMatAsCompCount()
            ChangePosOfCompAsPerOption()

            varCompTrans = swCompTrans

            swIDM = swADoc.InterferenceDetectionManager
            swIDM.TreatCoincidenceAsInterference = True
            isInterfering = swIDM.GetComponentsTransformInterference((varComp), (varCompTrans), outIntComp)

            Stop ' Observe transformation of objects for interference detection

            swIDM.Done() ' Interference detection stopped
            PrintInterferenceInfo()
            varComp = Nothing
        Else
            MsgBox("Select components")
        End If

    End Sub
    SetPosAsPerOption()
    Sub SetPosAsPerOption()
        If (userOpt = 0) Then
            gposX = 100.0 / 1000.0
            gposY = -50.0 / 1000.0
        ElseIf (userOpt = 1) Then
            gposX = 70.0 / 1000.0
            gposY = -50.0 / 1000.0
        End If
    End Sub
    Sub SelectAllComponent()
        Dim boolstatus As Boolean

```

```

boolstatus = swDoc.Extension.SelectByID2("SquarePad_Red-1@InterferenceAssem", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Green-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Blue-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Orange-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_SkyBlue-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Yellow-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("SquarePad_Pink-1@InterferenceAssem", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
End Sub
Sub PrintInterferenceInfo()
    If (isInterfering) Then
        Debug.Print("Interfering components:")
        Debug.Print("")
        For I = 0 To UBound(outIntComp)
            Dim swOutComp As Component2
            swOutComp = outIntComp(I)
            Debug.Print(swOutComp.Name2)
            swOutComp.Select4(True, Nothing, False)
            swOutComp = Nothing
        Next I
    Else
        Debug.Print("No interference")
    End If
End Sub
Sub getSelectedComp()
    ReDim swComp(numOfComp - 1)
    For I = 1 To numOfComp
        swComp(I - 1) = swSM.GetSelectedObject6(I, -1)
    Next I
    varComp = swComp
End Sub
Sub PrintComponentName(ByVal varComp As Object)
    If Not (IsNothing(varComp)) Then
        If userOpt = 0 Then
            Debug.Print("Detect interferences between coincident components:")
            Debug.Print("")
        Else
            Debug.Print("Detect interferences between intersecting components:")
            Debug.Print("")
        End If

        For I = LBound(varComp) To UBound(varComp)
            Dim swComp As Component2
            swComp = varComp(I)

            Debug.Print("Component name : " & swComp.Name2)
            swComp = Nothing
        Next I
    End If
    Debug.Print("")
End Sub
Sub CreateSameTransMatAsCompCount()
    ReDim swCompTrans(numOfComp - 1)
    If Not (IsNothing(varComp)) Then
        For I = LBound(varComp) To UBound(varComp)
            Dim swComp As Component2
            swComp = varComp(I)
            swCompTrans(I) = swComp.Transform2
            swComp = Nothing
        Next I
    End If
End Sub
Sub ChangePosOfCompAsPerOption()
    ChangeRefPosOfComp(gposX, gposY)
End Sub
Sub ChangeRefPosOfComp(ByVal posX As Double, ByVal posY As Double)
    If Not (IsNothing(varComp)) Then
        For I = LBound(varComp) To UBound(varComp)
            If (I > 0) Then
                Dim varData As Object
                varData = swCompTrans(I).ArrayData
                varData(9) = posX * I
                varData(10) = posY * I
                swCompTrans(I).ArrayData = varData
                If (userOpt = 2) Then
                    swComp(I).Transform2 = swCompTrans(I)
                    swDoc.ForceRebuild3(True)
                End If
            End If
        Next I
    End If
End Sub

Public swApp As SldWorks

End Class

```

### Determine if Display Dimension Marked for Drawing Example (VBA)

This example shows how to determine if a display dimension is marked to include in a drawing document.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly document.  
' 2. Insert a smart dimension and leave the PropertyManager  
'    page open so that the dimension is selected.  
' 3. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swDispDim As SldWorks.DisplayDimension  
    Dim swDim As SldWorks.Dimension  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swDispDim = swSelMgr.GetSelectedObject6(1, 0)  
    Set swDim = swDispDim.GetDimension  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "   Name of dimension = " & swDim.FullName  
    Debug.Print "   Marked for drawing? " & swDispDim.MarkedForDrawing  
  
End Sub
```

### Determine if Document is Dirty Example (VBA)

This example shows how to determine if a document is dirty (i.e., modified since opening it) and needs to be saved.

```
'-----  
' Preconditions:  
' 1. Open multiple model documents.  
' 2. Modify one of the model documents.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Traverses the open model documents and checks  
'    to see if they're dirty.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.GetFirstDocument  
  
    Do While Not swModel Is Nothing  
        Debug.Print swModel.GetPathName  
        Debug.Print "   Dirty? " & swModel.GetSaveFlag  
        Set swModel = swModel.GetNext  
    Loop  
  
End Sub
```

## Determine if Layer is Visible Example (VBA)

This example shows how to determine if a layer in a drawing is visible.

```
'-----
' Preconditions:
' 1. Open a drawing of a part.
' 2. Select a drawing view in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates a layer of the part in the selected drawing view.
' 2. Gets whether the layer is visible.
' 3. Examine the Immediate window and drawing.
'-----
Option Explicit

Private Sub ChangeComponentLayer _
(
    swApp As SldWorks.SldWorks, _
    swDraw As SldWorks.DrawingDoc, _
    sLayerName As String _
)
    Dim bRet As Boolean

    sLayerName = "Layer1"
    bRet = swDraw.CreateLayer2( _
        sLayerName, _
        "Layer for part in " & sLayerName, _
        0, swLineCONTINUOUS, swLW_NORMAL, True, True)

    ' Change in all views
    swDraw.ChangeComponentLayer sLayerName, True
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swView As SldWorks.View
    Dim swDrawModel As SldWorks.ModelDoc2
    Dim swDrawPart As SldWorks.PartDoc
    Dim vBody As Variant
    Dim swBody As SldWorks.Body2
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swLayerMgr As SldWorks.LayerMgr
    Dim swLayer As SldWorks.Layer
    Dim nErrors As Long
    Dim nWarnings As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    Set swDrawModel = swApp.OpenDoc6(swView.GetReferencedModelName, swDocPART, swOpenDocOptions_Silent, "", nErrors, nWarnings)
    Set swDrawPart = swDrawModel

    vBody = swDrawPart.GetBodies2(swSolidBody, True)

    Set swBody = vBody(0)
    Set swFace = swBody.GetFirstFace
    Set swEnt = swFace

    bRet = swView.SelectEntity(swEnt, False)

    ChangeComponentLayer swApp, swDraw, swView.Name

    Set swLayerMgr = swModel.GetLayerManager
    Set swLayer = swLayerMgr.GetLayer("Layer1")
    Debug.Print "Layer visible?" & swLayer.Visible
End Sub
```



### Determine if OLE Objects are Linked or Embedded Example (VBA)

This example shows how to determine if the OLE objects on a document are linked to external files or embedded in the document.

```
'-----  
' Preconditions:  
' 1. Open a model document that contains  
'    one or more OLE objects.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets whether each OLE object is linked  
'    or embedded in the model document.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim nCount As Long  
Dim ub As Long  
Dim oleobjoptions As Long  
Dim vOleObjs As Variant  
Dim i As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    Debug.Print "Number of OLE objects on document = " & swModelDocExt.GetOLEObjectCount(oleobjoptions)  
  
    vOleObjs = swModelDocExt.GetOLEObjects(oleobjoptions)  
    ub = UBound(vOleObjs)  
    For i = 0 To ub  
        Debug.Print "OLE object " & i & " linked (OLE object is embedded if False)? " & vOleObjs(i).IsLinked  
    Next i  
  
End Sub
```

## Determine if Sketch Contour or Sketch Region Example (VBA)

This example shows how to determine if the object is a sketch contour or sketch region. It also shows how to use the **TypeOf** keyword to determine the type of object in the first element in the array.

```
'-----
' Preconditions:
' 1. Open a part that contains a Boss-Extrude1 feature
'    with sketch contours and sketch regions.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens Boss-Extrude1's sketch and selects
'    the sketch contours and sketch regions.
' 2. Gets whether the first selection in the sketch
'    is a sketch contour or sketch region.
' 3. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.PartDoc
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    boolstatus = Part.Extension.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)

    'Boss-Extrude1 has sketch contours and sketch regions
    Dim Extrudefeature As SldWorks.Feature
    Dim Extrudefeature_DEF As SldWorks.ExtrudeFeatureData2
    Dim SelMgr As SldWorks.SelectionMgr
    Dim ContourArray As Variant

    Set SelMgr = Part.SelectionManager
    Set Extrudefeature = SelMgr.GetSelectedObject5(1)
    Set Extrudefeature_DEF = Extrudefeature.GetDefinition
    Extrudefeature_DEF.AccessSelections Part, Nothing
    ContourArray = Extrudefeature_DEF.Contours
    Extrudefeature_DEF.ReleaseSelectionAccess

    Dim SR As SketchRegion
    Dim SC As SketchContour

    ContourArray(0).Select False, 0
    Debug.Print TypeName(ContourArray(0))
    If TypeOf ContourArray(0) Is SketchRegion Then
        Debug.Print "Object is SketchRegion"
        Set SR = ContourArray(0)
    Else
        Debug.Print "Object is SketchContour"
        Set SC = ContourArray(0)
    End If
End Sub
```

### Determine if Sketch is Shared Example (VBA)

This example shows how to determine whether or not a sketch is shared with multiple features.

```
'-----
' Preconditions:
' 1. Open a part, assembly, or drawing.
' 2. Select a sketch used to create a feature.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets whether the sketch for the selected
'    feature is shared.
' 2. Gets the children of the feature.
' 3. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim i As Long
    Dim bRet As Boolean
    Dim vChildArr As Variant
    Dim vChild As Variant
    Dim swChildFeat As SldWorks.Feature

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    Set swSketch = swFeat.GetSpecificFeature2

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Sketch = " & swFeat.Name
    Debug.Print " IsShared = " & swSketch.IsShared

    vChildArr = swFeat.GetChildren: If IsEmpty(vChildArr) Then Exit Sub
    For Each vChild In vChildArr
        Set swChildFeat = vChild
        Debug.Print " " & swChildFeat.Name & " [" & swChildFeat.GetTypeName & "]"
    Next vChild

End Sub
```

## Determine if Sketch Suitable for Feature Example (VBA)

This example shows how to determine if a sketch is suitable for use in a feature.

**NOTE:** Sketch geometry can be used in a variety of ways in SOLIDWORKS Design. However, not every sketch is suitable for every purpose. For example, (non-thin) extrusions require a closed profile.

```

'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Select a sketch.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Checks the selected sketch.
' 2. Examine the Immediate window.
'-----

Option Explicit

' List of feature types to be used with the ISketch::CheckFeatureUse
Public Enum swSketchCheckFeatureProfileUsage_e
    swSketchCheckFeature_UNSET = 0
    swSketchCheckFeature_BASEEXTRUDE = 1
    swSketchCheckFeature_BASEEXTRUDETHIN = 2
    swSketchCheckFeature_BOSSEXTRUDE = 3
    swSketchCheckFeature_BOSSEXTRUDETHIN = 4
    swSketchCheckFeature_SURFACEEXTRUDE = 5
    swSketchCheckFeature_BASEREVOLVE = 6
    swSketchCheckFeature_BASEREVOLVEHIN = 7
    swSketchCheckFeature_BOSSREVOLVE = 8
    swSketchCheckFeature_BOSSREVOLVEHIN = 9
    swSketchCheckFeature_SURFACEREVOLVE = 10
    swSketchCheckFeature_CUTEXTRUDE = 11
    swSketchCheckFeature_CUTEXTRUDETHIN = 12
    swSketchCheckFeature_CUTREVOLVE = 13
    swSketchCheckFeature_CUTREVOLVEHIN = 14
    swSketchCheckFeature_SWEEPSECTION = 15
    swSketchCheckFeature_SURFACESWEEPSECTION = 16
    swSketchCheckFeature_SWEEPPATHORGUIDE = 17
    swSketchCheckFeature_LOFTSECTION = 18
    swSketchCheckFeature_SURFACELOFTSECTION = 19
    swSketchCheckFeature_LOFTGUIDE = 20
    swSketchCheckFeature_RIBSECTION = 21
    swSketchCheckFeature_SHEETMETAL_BASEFLANGE = 22
End Enum

' A list of return status values for the ISketch::CheckFeatureUse
Public Enum swSketchCheckFeatureStatus_e
    swSketchCheckFeatureStatus_UnknownError = -1
    swSketchCheckFeatureStatus_OK = 0
    swSketchCheckFeatureStatus_EntXEnt = 1
    swSketchCheckFeatureStatus_EntXSelf = 2
    swSketchCheckFeatureStatus_EntUnspecBad = 3
    swSketchCheckFeatureStatus_ThreeEnts = 4
    swSketchCheckFeatureStatus_EmptySketch = 5
    swSketchCheckFeatureStatus_WrongOpen = 6
    swSketchCheckFeatureStatus_WrongManyContours = 7
    swSketchCheckFeatureStatus_ZeroLengthEnt = 8
    swSketchCheckFeatureStatus_ManyOpen = 9
    swSketchCheckFeatureStatus_NoOpen = 10
    swSketchCheckFeatureStatus_MixedContours = 11
    swSketchCheckFeatureStatus_CturXCtur = 12
    swSketchCheckFeatureStatus_DisjCturs = 13
    swSketchCheckFeatureStatus_OpenWantClosed = 14
    swSketchCheckFeatureStatus_ClosedWantOpen = 15
    swSketchCheckFeatureStatus_DoubleContainment = 16
    swSketchCheckFeatureStatus_MoreThanOneContour = 17
    swSketchCheckFeatureStatus_OneOpenContourExpected = 18
    swSketchCheckFeatureStatus_OneClosedContourExpected = 19
    swSketchCheckFeatureStatus_WantSingleOpenOrMultiClosedDisjoint = 20
    swSketchCheckFeatureStatus_NeedsAxis = 21
    swSketchCheckFeatureStatus_OpenOrUnclear = 22
    swSketchCheckFeatureStatus_ContourIntersectsCenterLine = 23
End Enum

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim nRetVal As Long
    Dim nOpenCount As Long
    Dim nClosedCount As Long
    Dim i As Long

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    Set swSketch = swFeat.GetSpecificFeature2

    Debug.Print "Feature = " & swFeat.Name

    For i = 0 To 22
        nRetVal = swSketch.CheckFeatureUse(i, nOpenCount, nClosedCount)
    
```

```
Debug.Print " FeatCheckType = " & i  
Debug.Print " RetVal      = " & nRetVal  
Debug.Print " OpenCount   = " & nOpenCount  
Debug.Print " ClosedCount = " & nClosedCount  
Debug.Print ""
```

```
Next i
```

```
Debug.Print ""
```

```
End Sub
```

## Determine Type of Face Example (VBA)

This example shows how to identify holes, bosses, and tangent faces on a selected face.

**NOTE:** To determine the type of face, start from a selected face and use `IFace2::GetFirstLoop` to get the first loop in the face. For every loop on the face (use `!Loop2::IsOuter` equals false), then get the coedge for this loop and the partner coedge for the coedge.

For each coedge, get the face normal at the middle of the coedge. This is found by obtaining the middle of the coedge with a call to `ICoEdge::GetCurveParams`, and a call to `ICoEdge::Evaluate` on the midpoint of the coedge to get the point's coordinates.

The normal is returned by a call to `ISurface::EvaluateAtPoint` at the point in the middle of the coedge. If the normals for the two faces are equal, then there is a tangent face (fillet) on the selected face. If the normals are not equal, then if the cross product of the two normals is equal to the tangent of the face, there is a hole at the coedge. Otherwise, if there is a loop that is not the outer loop that does not meet any of the other conditions, it is a boss.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\block.sldprt.
' 2. Select either face that contains a hole.
'
' Postconditions:
' 1. Identifies the hole in the selected face
'    and displays a message box.
' 2. Click OK to close the message box.
'
' NOTE: If you run this macro and select a face
' that does not have a hole, boss, or tangent face,
' then a message box is not displayed.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swDoc As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace As SldWorks.Face2
    Dim swSelData As SldWorks.SelectData

    Set swApp = GetObject(, "SldWorks.Application")
    Set swDoc = swApp.ActiveDoc

    'Verify that a face is selected
    Set swSelMgr = swDoc.SelectionManager
    If swSelMgr.GetSelectedObjectCount <> 1 Then Exit Sub
    If swSelMgr.GetSelectedObjectType2(1) <> swSelFACES Then Exit Sub

    'Get the selected face
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    Set swSelData = swSelMgr.CreateSelectData

    'Find the holes and bosses in the selected face
    Call FindHolesAndBosses(swFace, swSelData)

End Sub

'This subroutine locates all holes, bosses,
'and tangent faces in the selected face
Sub FindHolesAndBosses(swFace, swSelData)
    Dim swThisLoop As SldWorks.Loop2
    Dim swThisCoEdge As SldWorks.CoEdge
    Dim swPartnerFace As SldWorks.Face2
    Dim swPartnerLoop As SldWorks.Loop2
    Dim swPartnerCoEdge As SldWorks.CoEdge
    Dim swEntity As SldWorks.Entity
    Dim varThisNormal As Variant
    Dim varPartnerNormal As Variant
    Dim varCrossProduct As Variant
    Dim varTangent As Variant

    Set swThisLoop = swFace.GetFirstLoop

    'For every loop on the face
    Do While Not swThisLoop.IsNothing
        If swThisLoop.IsOuter = False Then
            Set swThisCoEdge = swThisLoop.GetFirstCoEdge
            Set swPartnerCoEdge = swThisCoEdge.GetPartner
            varThisNormal = GetFaceNormalAtMidCoEdge(swThisCoEdge)
            varPartnerNormal = GetFaceNormalAtMidCoEdge(swPartnerCoEdge)
            If VectorsAreEqual(varThisNormal, varPartnerNormal) Then
                ' The neighboring face is tangent to the
                ' selected face; additional code is necessary for
                ' determining how the surface moves away from the
                ' selected face
                Set swPartnerLoop = swPartnerCoEdge.GetLoop
                Set swPartnerFace = swPartnerLoop.GetFace
                Set swEntity = swPartnerFace
                swEntity.Select4 False, swSelData
                MsgBox "This face is tangent."
            Else
                ' There is a sufficient change between
                ' the two faces to determine the kind of transition
                ' being made
                varCrossProduct = GetCrossProduct(varThisNormal, varPartnerNormal)
                varTangent = GetTangentAtMidCoEdge(swThisCoEdge)
                If VectorsAreEqual(varCrossProduct, varTangent) Then
                    'Hole
                    Set swPartnerLoop = swPartnerCoEdge.GetLoop
                    Set swPartnerFace = swPartnerLoop.GetFace
                    Set swEntity = swPartnerFace
                End If
            End If
        End If
        Set swThisLoop = swThisLoop.GetNext
    Loop
End Sub
```

```

        swEntity.Select4 False, swSelData
        MsgBox "This face is a hole."
    Else
        'Boss
        Set swPartnerLoop = swPartnerCoEdge.GetLoop
        Set swPartnerFace = swPartnerLoop.GetFace
        Set swEntity = swPartnerFace
        swEntity.Select4 False, swSelData
        MsgBox "This face is a boss."
    End If
End If
End If
Set swThisLoop = swThisLoop.GetNext
Loop
End Sub

' This function returns the face normal from the provided coedge
Function GetFaceNormalAtMidCoEdge(swCoEdge As CoEdge) As Variant
    Dim swFace As SldWorks.Face2
    Dim swSurface As SldWorks.Surface
    Dim swLoop As SldWorks.Loop2
    Dim varParams As Variant
    Dim varPoint As Variant
    Dim dblMidParam As Double
    Dim dblNormal(2) As Double
    Dim bFaceSenseReversed As Boolean

    varParams = swCoEdge.GetCurveParams
    If varParams(6) > varParams(7) Then
        dblMidParam = (varParams(6) - varParams(7)) / 2 + varParams(7)
    Else
        dblMidParam = (varParams(7) - varParams(6)) / 2 + varParams(6)
    End If
    varPoint = swCoEdge.Evaluate(dblMidParam)

    ' Get the face of the given coedge; remember to check
    ' for the sense of the face
    Set swLoop = swCoEdge.GetLoop
    Set swFace = swLoop.GetFace
    Set swSurface = swFace.GetSurface
    bFaceSenseReversed = swFace.FaceInSurfaceSense
    varParams = swSurface.EvaluateAtPoint(varPoint(0), varPoint(1), varPoint(2))
    If bFaceSenseReversed Then
        ' Negate the surface normal as it is opposite from the face normal
        dblNormal(0) = -varParams(0)
        dblNormal(1) = -varParams(1)
        dblNormal(2) = -varParams(2)
    Else
        dblNormal(0) = varParams(0)
        dblNormal(1) = varParams(1)
        dblNormal(2) = varParams(2)
    End If
    GetFaceNormalAtMidCoEdge = dblNormal
End Function

' Get the tangent vector for the provided coedge
Function GetTangentAtMidCoEdge(swCoEdge As Variant) As Variant
    Dim varParams As Variant
    Dim dblMidParam As Double
    Dim dblTangent(2) As Double

    varParams = swCoEdge.GetCurveParams
    If varParams(6) > varParams(7) Then
        dblMidParam = (varParams(6) - varParams(7)) / 2 + varParams(7)
    Else
        dblMidParam = (varParams(7) - varParams(6)) / 2 + varParams(6)
    End If

    varParams = swCoEdge.Evaluate(dblMidParam)
    dblTangent(0) = varParams(3)
    dblTangent(1) = varParams(4)
    dblTangent(2) = varParams(5)
    GetTangentAtMidCoEdge = dblTangent
End Function

' Obtain the cross product of two vectors
Function GetCrossProduct(varVec1 As Variant, varVec2 As Variant) As Variant
    Dim dblCross(2) As Double

    dblCross(0) = varVec1(1) * varVec2(2) - varVec1(2) * varVec2(1)
    dblCross(1) = varVec1(2) * varVec2(0) - varVec1(0) * varVec2(2)
    dblCross(2) = varVec1(0) * varVec2(1) - varVec1(1) * varVec2(0)
    GetCrossProduct = dblCross
End Function

' Checks whether two vectors are equal within a tolerance of 1.0 e^-10
Function VectorsAreEqual(varVec1 As Variant, varVec2 As Variant) As Boolean
    Dim dblMag As Double
    Dim dblDot As Double
    Dim dblUnit1(2) As Double
    Dim dblUnit2(2) As Double

    dblMag = (varVec1(0) * varVec1(0) + varVec1(1) * varVec1(1) + varVec1(2) * varVec1(2)) ^ 0.5
    dblUnit1(0) = varVec1(0) / dblMag: dblUnit1(1) = varVec1(1) / dblMag: dblUnit1(2) = varVec1(2) / dblMag
    dblMag = (varVec2(0) * varVec2(0) + varVec2(1) * varVec2(1) + varVec2(2) * varVec2(2)) ^ 0.5
    dblUnit2(0) = varVec2(0) / dblMag: dblUnit2(1) = varVec2(1) / dblMag: dblUnit2(2) = varVec2(2) / dblMag
    dblDot = dblUnit1(0) * dblUnit2(0) + dblUnit1(1) * dblUnit2(1) + dblUnit1(2) * dblUnit2(2)
    dblDot = Abs(dblDot - 1#)

    If dblDot < 0.0000000001 Then '1.0e-10
        VectorsAreEqual = True
    Else
        VectorsAreEqual = False
    End If
End Function

```

```
End If  
End Function
```



Differences Between Unmanaged C++ and C++/CLI Code

Many differences exist between unmanaged C++ and .NET-based C++/CLI (also called managed C++). One difference between unmanaged C++ and C++/CLI is the syntax. Although this Help shows the syntax for C++/CLI only, SOLIDWORKS API supports both unmanaged C++ and C++/CLI.

Examine the examples in the following table. The same example is written in unmanaged C++ and C++/CLI code.

| Example                 | Code                                                                                                                                                                                                                                            |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Traverse Features       | <ul style="list-style-type: none"><li>• <a href="#">Unmanaged C++</a></li><li>• <a href="#">C++/CLI</a></li></ul>                                                                                                                               |
| Traverse Configurations | <ul style="list-style-type: none"><li>• <a href="#">Unmanaged C++ (demonstrates a BSTR* C-array)</a></li><li>• <a href="#">Unmanaged C++ (demonstrates a VARIANT array)</a></li><li>• <a href="#">C++/CLI (demonstrates an array)</a></li></ul> |
| Select Objects          | <ul style="list-style-type: none"><li>• <a href="#">Unmanaged C++</a></li><li>• <a href="#">C++/CLI</a></li></ul>                                                                                                                               |

To learn more about the differences between unmanaged C++ and C++/CLI, SOLIDWORKS recommends that you consult the internet and books containing information about their differences.

### Dimension Edge in Drawing Example (VBA)

This examples shows how to dimension an edge in a drawing view.

```
'-----
'
' Problem:
'   This example shows how to automatically add a
'   dimension to a straight edge in all drawing views
'   in which it appears. The edge geometry is transformed
'   to the space of each drawing view and, depending on
'   whether it is horizontal or vertical, an appropriate
'   style of dimension is added.
'
'   This example could form the basis for an application
'   to automatically dimension a model when it is added to
'   a drawing.
'
' Preconditions:
'   (1) Part or assembly is open.
'   (2) If an assembly, it is fully resolved.
'   (3) A straight edge is selected in the SOLIDWORKS Design graphics area.
'
' Postconditions:
'   (1) New drawing is created with three views.
'   (2) If possible, the previously selected edge
'       is dimensioned in all drawing views.
'
' NOTE: The dimension is not created if
'       the edge cannot be converted in a drawing view.
'-----

Option Explicit
Public Const LINE_TYPE      As Integer = 3001
Public Const CIRCLE_TYPE    As Integer = 3002
Public Const ELLIPSE_TYPE   As Integer = 3003
Public Const INTERSECTION_TYPE As Integer = 3004
Public Const BCURVE_TYPE    As Integer = 3005
Public Const SPCURVE_TYPE   As Integer = 3006
Public Const CONSTPARAM_TYPE As Integer = 3008
Public Const TRIMMED_TYPE   As Integer = 3009

' Define two types
Type DoubleRec
    dValue As Double
End Type
Type Long2Rec
    iLower As Long
    iUpper As Long
End Type

' Extract two integer values out of a single double value,
' by assigning a DoubleRec to the double value and
' copying the value over an Long2Rec and
' extracting the integer values.
Function ExtractFields _
(
    ByVal dValue As Double, _
    iLower As Long, _
    iUpper As Long _
)
    Dim dr          As DoubleRec
    Dim i2r         As Long2Rec

    ' Set the double value
    dr.dValue = dValue

    ' Copy the values
    LSet i2r = dr
```

```

' Extract the values
iLower = i2r.iLower
iUpper = i2r.iUpper
End Function

Sub main()

Const sPathToTemplate As String = "c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\templates\drawing.drt.dot"

Const nTolerance As Double = 0.00000001

Const nXoffset As Double = 0.01
Const nYoffset As Double = 0.01

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swEdge As SldWorks.Edge
Dim swEnt As SldWorks.entity
Dim swCurve As SldWorks.Curve
Dim vCurveParam As Variant
Dim nDummy As Long
Dim nIdentity As Long
Dim nTag As Long
Dim nSense As Long

Dim swMathUtil As SldWorks.MathUtility
Dim nPtData(2) As Double
Dim vPtData As Variant
Dim swModelStartPt As SldWorks.MathPoint
Dim swModelEndPt As SldWorks.MathPoint
Dim swViewStartPt As SldWorks.MathPoint
Dim swViewEndPt As SldWorks.MathPoint

Dim swDraw As SldWorks.DrawingDoc
Dim swDrawModel As SldWorks.ModelDoc2
Dim swView As SldWorks.view
Dim swViewXform As SldWorks.MathTransform
Dim vOutline As Variant
Dim swDispDim As SldWorks.DisplayDimension

Dim nXpos As Double
Dim nYpos As Double
Dim bRet As Boolean
Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swEdge = swSelMgr.GetSelectedObject5(1)
Set swCurve = swEdge.GetCurve
Set swEnt = swEdge

vCurveParam = swEdge.GetCurveParams2
ExtractFields vCurveParam(8), nDummy, nIdentity
ExtractFields vCurveParam(9), nDummy, nTag
ExtractFields vCurveParam(10), nDummy, nSense

Debug.Print "Start = (" & vCurveParam(0) * 1000# & ", " & vCurveParam(1) * 1000# & ", " & vCurveParam(2) * 1000# & ") mm "
Debug.Print "End = (" & vCurveParam(3) * 1000# & ", " & vCurveParam(4) * 1000# & ", " & vCurveParam(5) * 1000# & ") mm "
Debug.Print "Uparm = [" & vCurveParam(6) & ", " & vCurveParam(7) & "]"
Debug.Print "Identity = " & nIdentity
Debug.Print "Tag = " & nTag
Debug.Print "Sense = " & nSense

' Derived quantity
Debug.Print "Length = " & swCurve.GetLength2(vCurveParam(6), vCurveParam(7)) * 1000# & " mm "
Debug.Print ""

' Only makes sense for straight edges
If LINE_TYPE <> nIdentity Then Exit Sub

Set swMathUtil = swApp.GetMathUtility

```

```

nPtData(0) = vCurveParam(0)
nPtData(1) = vCurveParam(1)
nPtData(2) = vCurveParam(2)
vPtData = nPtData
Set swModelStartPt = swMathUtil.CreatePoint(vPtData)

nPtData(0) = vCurveParam(3)
nPtData(1) = vCurveParam(4)
nPtData(2) = vCurveParam(5)
vPtData = nPtData
Set swModelEndPt = swMathUtil.CreatePoint(vPtData)

' Start creating drawing of the model
Set swDraw = swApp.NewDocument("C:\Program Files\SOLIDWORKS\data\templates\drawing.drwdot", swDwgPaperASize, 0, 0)
Set swDrawModel = swDraw

bRet = swDraw.Create3rdAngleViews2(swModel.GetPathName)
Debug.Assert bRet

Set swView = swDraw.GetFirstView
Set swView = swView.GetNextView
Do While Not swView Is Nothing
    ' Select regardless
    bRet = swView.SelectEntity(swEnt, False)
    Debug.Assert bRet

    vOutline = swView.GetOutline

    Set swViewXform = swView.ModelToViewTransform
    Set swViewStartPt = swModelStartPt.MultiplyTransform(swViewXform)
    Set swViewEndPt = swModelEndPt.MultiplyTransform(swViewXform)

    Debug.Print "View      = " & swView.Name
    Debug.Print "Start    = (" & swViewStartPt.ArrayData(0) * 1000# & ", " & swViewStartPt.ArrayData(1) * 1000# & ", " & swViewStartPt.ArrayData(2) * 1000# & ") mm "
    Debug.Print "End      = (" & swViewEndPt.ArrayData(0) * 1000# & ", " & swViewEndPt.ArrayData(1) * 1000# & ", " & swViewEndPt.ArrayData(2) * 1000# & ") mm "
    Debug.Print ""

    If Abs(swViewStartPt.ArrayData(0) - swViewEndPt.ArrayData(0)) < nTolerance Then
        ' Must be vertical
        ' Place dimension midway up edge and to the right of view
        nXpos = vOutline(0) - nXOffset
        nYpos = Abs((swViewStartPt.ArrayData(1) + swViewEndPt.ArrayData(1)) / 2#)

        ' NULL if cannot convert edge in this view
        Set swDispDim = swDrawModel.AddVerticalDimension2(nXpos, nYpos, 0#)
    ElseIf Abs(swViewStartPt.ArrayData(1) - swViewEndPt.ArrayData(1)) < nTolerance Then
        ' Must be horizontal
        ' Place dimension midway across edge and above view
        nXpos = Abs((swViewStartPt.ArrayData(0) + swViewEndPt.ArrayData(0)) / 2#)
        nYpos = vOutline(3) + nYOffset

        ' NULL if cannot convert edge in this view
        Set swDispDim = swDrawModel.AddHorizontalDimension2(nXpos, nYpos, 0#)
    Else
        ' Neither horizontal or vertical
        ' Place dimension near middle of edge
        nXpos = Abs((swViewStartPt.ArrayData(0) + swViewEndPt.ArrayData(0)) / 2#) + nXOffset
        nYpos = Abs((swViewStartPt.ArrayData(1) + swViewEndPt.ArrayData(1)) / 2#) + nYOffset

        ' Depends on the orientation of the entity in the drawing view,
        ' thus, could be NULL
        '
        ' Create the dimension even if the entity is not
        ' visible in the drawing view
        Set swDispDim = swDrawModel.AddDimension2(nXpos, nYpos, 0#)
    End If

    Set swView = swView.GetNextView
Loop
End Sub

'-----

```

## Disable Cell Drop-down Lists in Design Table Example (C#)

This example shows how to disable drop-down lists for cells in a design table.

```
//-----
// Preconditions:
// 1. Open a part that has a design table.
// 2. In the SolidWorks Visual Studio for Applications Tools
// IDE, click Project > Add Reference > .NET >
// Microsoft.Office.Interop.Excel > OK.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets and edits the design table.
// 2. Disables drop-down lists for cells in the design table.
// 3. Updates and closes the design table.
// 4. Examine the Immediate window.
//
// NOTE: You can also expand Tables on the ConfigurationManager
// tab, right-click Design Table, and click Edit Feature to
// verify that Enable cell drop-down lists (may be slower)
// is not selected.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DesignTable swDesignTable = default(DesignTable);
            bool status = false;
            Microsoft.Office.Interop.Excel.Worksheet myWorksheet = default(Microsoft.Office.Interop.Excel.Worksheet);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Get and edit design table
            swDesignTable = (DesignTable)swModel.GetDesignTable();
            myWorksheet = (Microsoft.Office.Interop.Excel.Worksheet)swDesignTable.EditTable2(false);

            //Allow changes to characteristics of design table
            swDesignTable.EditFeature();

            //Disable cell drop-down lists
            status = swDesignTable.EnableCellDropdownLists;
            Debug.Print("Current \"Enable cell drop-down lists (may be slower)\" setting: " + status);
            swDesignTable.EnableCellDropdownLists = false;
            status = swDesignTable.EnableCellDropdownLists;
            Debug.Print("Modified \"Enable cell drop-down lists (may be slower)\" setting: " + status);

            status = swDesignTable.UpdateFeature();
            Debug.Print("Design table feature updated? " + status);

            //Update and close design table
            swDesignTable.UpdateTable((int)swDesignTable.UpdateOptions_e.swUpdateDesignTableAll, true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

### Disable Cell Drop-down Lists in Design Table Example (VBA)

This example shows how to disable drop-down lists for cells in a design table.

```
'-----
' Preconditions:
' 1. Open a part that has a design table.
' 2. In the Microsoft Visual Basic for Applications IDE, click
'    Tools > References > Microsoft Excel release Object Library > OK.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets and edits the design table.
' 2. Disables drop-down lists for cells in the design table.
' 3. Updates and closes the design table.
' 4. Examine the Immediate window.
'
' NOTE: You can also expand Tables on the ConfigurationManager
' tab, right-click Design Table, and click Edit Feature to
' verify that Enable cell drop-down lists (may be slower)
' is not selected.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDesignTable As SldWorks.DesignTable
    Dim status As Boolean
    Dim myWorksheet As Excel.worksheet

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Get and edit design table
    Set swDesignTable = swModel.GetDesignTable
    Set myWorksheet = swDesignTable.EditTable2(False)

    'Allow changes to characteristics of design table
    swDesignTable.EditFeature

    'Disable cell drop-down lists
    status = swDesignTable.EnableCellDropdownLists
    Debug.Print "Current ""Enable cell drop-down lists (may be slower)"" setting: " & status
    swDesignTable.EnableCellDropdownLists = False
    status = swDesignTable.EnableCellDropdownLists
    Debug.Print "Modified ""Enable cell drop-down lists (may be slower)"" setting: " & status

    status = swDesignTable.UpdateFeature
    Debug.Print "Design table feature updated? " & status

    'Update and close design table
    swDesignTable.UpdateTable swDesignTableUpdateOptions_e.swUpdateDesignTableAll, True

End Sub
```

## Disable Cell Drop-down Lists in Design Table Example (VB.NET)

This example shows how to disable drop-down lists for cells in a design table.

```

-----
Preconditions:
1. Open a part that has a design table.
2. In the SolidWorks Visual Studio for Applications Tools
   IDE, click Project > Add Reference > .NET >
   Microsoft.Office.Interop.Excel > OK.
3. Open the Immediate window.
Postconditions:
1. Gets and edits the design table.
2. Disables drop-down lists for cells in the design table.
3. Updates and closes the design table.
4. Examine the Immediate window.
NOTE: You can also expand Tables on the ConfigurationManager
tab, right-click Design Table, and click Edit Feature to
verify that Enable cell drop-down lists (may be slower)
is not selected.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDesignTable As DesignTable
        Dim status As Boolean
        Dim myWorksheet As Microsoft.Office.Interop.Excel.Worksheet

        swModel = swApp.ActiveDoc

        'Get and edit design table
        swDesignTable = swModel.GetDesignTable
        myWorksheet = swDesignTable.EditTable2(False)

        'Allow changes to characteristics of design table
        swDesignTable.EditFeature

        'Disable cell drop-down lists
        status = swDesignTable.EnableCellDropdownLists
        Debug.Print("Current " & "Enable cell drop-down lists (may be slower)" & " setting: " & status)
        swDesignTable.EnableCellDropdownLists = False
        status = swDesignTable.EnableCellDropdownLists
        Debug.Print("Modified " & "Enable cell drop-down lists (may be slower)" & " setting: " & status)

        status = swDesignTable.UpdateFeature
        Debug.Print("Design table feature updated? " & status)

        'Update and close design table
        swDesignTable.UpdateTable(swDesignTable.UpdateOptions_e.swUpdateDesignTableAll, True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

## Disable Equation Example (C#)

This example shows how to disable an equation.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\partequations.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Disables an equation.
// 2. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            EquationMgr swEqnMgr = default(EquationMgr);
            int i = 0;
            int count = 0;
            int disabledCount = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swEqnMgr = (EquationMgr)swModel.GetEquationMgr();
            count = swEqnMgr.GetCount();
            Debug.Print("Number of equations in part = " + count);
            for (i = 0; i < count; i++)
            {
                Debug.Print("  Eqn(" + i + ") = " + swEqnMgr.get_Equation(i));
            }

            Debug.Print("");

            disabledCount = swEqnMgr.GetDisabledEquationCount();
            Debug.Print("Number of disabled equations in part before running macro = " + disabledCount);

            swEqnMgr.set_Disabled((count - 1), true);
            Debug.Print("Eqn(" + (count - 1) + ") disabled? " + swEqnMgr.get_Disabled(count - 1));

            disabledCount = swEqnMgr.GetDisabledEquationCount();
            Debug.Print("Number of disabled equations in part after running macro = " + disabledCount);

            Debug.Print("");

            count = swEqnMgr.GetCount();
            Debug.Print("Number of equations in part = " + count);
            for (i = 0; i < count; i++)
            {
                Debug.Print("  Eqn(" + i + ") = " + swEqnMgr.get_Equation(i));
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```



### Disable Equation Example (VBA)

This example shows how to disable an equation.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\partequations.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Disables an equation.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swEqnMgr As SldWorks.EquationMgr
    Dim i As Long
    Dim count As Long
    Dim disabledCount As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swEqnMgr = swModel.GetEquationMgr
    count = swEqnMgr.GetCount
    Debug.Print "Number of equations in part = " & count
    For i = 0 To count - 1
        Debug.Print "  Eqn(" & i & ") = " & swEqnMgr.Equation(i)
    Next i

    Debug.Print ""

    disabledCount = swEqnMgr.GetDisabledEquationCount
    Debug.Print "Number of disabled equations in part before running macro = " & disabledCount

    swEqnMgr.Disabled(count - 1) = True
    Debug.Print "Eqn(" & (count - 1) & ") disabled? " & swEqnMgr.Disabled(count - 1)

    disabledCount = swEqnMgr.GetDisabledEquationCount
    Debug.Print "Number of disabled equations in part after running macro = " & disabledCount

    Debug.Print ""

    count = swEqnMgr.GetCount
    Debug.Print "Number of equations in part = " & count
    For i = 0 To count - 1
        Debug.Print "  Eqn(" & i & ") = " & swEqnMgr.Equation(i)
    Next i

End Sub
```

## Disable Equation Example (VB.NET)

This example shows how to disable an equation.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\api\partequations.sldprt.
2. Open the Immediate window.
Postconditions:
1. Disables an equation.
2. Examine the Immediate window.
NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swEqnMgr As EquationMgr
        Dim i As Integer
        Dim count As Integer
        Dim disabledCount As Integer

        swModel = swApp.ActiveDoc
        swEqnMgr = swModel.GetEquationMgr
        count = swEqnMgr.GetCount
        Debug.Print("Number of equations in part = " & count)
        For i = 0 To count - 1
            Debug.Print(" Eqn(" & i & ") = " & swEqnMgr.Equation(i))
        Next i

        Debug.Print("")

        disabledCount = swEqnMgr.GetDisabledEquationCount
        Debug.Print("Number of disabled equations in part before running macro = " & disabledCount)

        swEqnMgr.Disabled(count - 1) = True
        Debug.Print("Eqn(" & (count - 1) & ") disabled? " & swEqnMgr.Disabled(count - 1))

        disabledCount = swEqnMgr.GetDisabledEquationCount
        Debug.Print("Number of disabled equations in part after running macro = " & disabledCount)

        Debug.Print("")

        count = swEqnMgr.GetCount
        Debug.Print("Number of equations in part = " & count)
        For i = 0 To count - 1
            Debug.Print(" Eqn(" & i & ") = " & swEqnMgr.Equation(i))
        Next i

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

## Disable Selection of Faces and Edges Using a Pre-Notify Event Example (C#)

This example shows how to disable the interactive selection of these entities using a pre-notify event:

- faces in part and assembly documents
- edges in drawing documents

```
//-----
// Preconditions: Open a part, assembly, or drawing.
//
// NOTE: Tools > Options > System > Stop VSTA debugger
// on macro exit must be cleared for this macro
// to run to completion.
//
// Postconditions:
// 1. Disables interactively selecting faces in
//    a part or assembly.
//    - or -
//    Disables interactively selecting edges in a
//    drawing.
// 2. Click the Stop Debugging button in the
//    SOLIDWORKS Visual Studio Tools for
//    Applications IDE to re-enable the
//    interactive selection of faces in
//    a part or assembly document or edges in
//    a drawing document.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Collections;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc pDoc;
        public AssemblyDoc aDoc;
        public DrawingDoc dDoc;
        public void Main()
        {
            ModelDoc2 swModel;
            Hashtable openPart;
            Hashtable openAssembly;
            Hashtable openDrawing;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Determine the document type
            // and set up event handlers
            if (swModel.GetType() == (int)swDocumentTypes_e.swDocPART)
            {
                pDoc = (PartDoc)swModel;
                openPart = new Hashtable();
            }
            else if (swModel.GetType() == (int)swDocumentTypes_e.swDocASSEMBLY)
            {
                aDoc = (AssemblyDoc)swModel;
                openAssembly = new Hashtable();
            }
            else if (swModel.GetType() == (int)swDocumentTypes_e.swDocDRAWING)
            {
                dDoc = (DrawingDoc)swModel;
                openDrawing = new Hashtable();
            }
            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            if ((pDoc != null))
            {
                pDoc.UserSelectionPreNotify += this.pDoc_UserSelectionPreNotify;
            }
            if ((aDoc != null))
            {
                aDoc.UserSelectionPreNotify += this.aDoc_UserSelectionPreNotify;
            }
            if ((dDoc != null))
            {
                dDoc.UserSelectionPreNotify += this.dDoc_UserSelectionPreNotify;
            }
        }

        private int pDoc_UserSelectionPreNotify(int SelectionType)
        {
            int functionReturnValue = 0;
            // Disable the selection of faces in this part
            if (SelectionType == (int)swSelectType_e.swSelFACES)
            {
                functionReturnValue = 1;
            }
        }
    }
}
```

```

        return functionReturnValue;
    }

    public int aDoc_UserSelectionPreNotify(int SelectionType)
    {
        int functionReturnValue = 0;
        // Disable the selection of faces in this assembly
        if (SelectionType == (int)swSelectType_e.swSelFACES)
        {
            functionReturnValue = 1;
        }
        return functionReturnValue;
    }

    private int dDoc_UserSelectionPreNotify(int SelectionType)
    {
        int functionReturnValue = 0;
        // Disable the selection of edges in this drawing
        if (SelectionType == (int)swSelectType_e.swSelEDGES)
        {
            functionReturnValue = 1;
        }
        return functionReturnValue;
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

### Disable Selection of Faces and Edges Using a Pre-Notify Event Example (VBA)

This example shows how to disable the interactive selection of these entities using a pre-notify event:

- faces in part and assembly documents
- edges in drawing documents

```
'-----
' Preconditions:
' 1. Copy Module into the macro.
' 2. Click Insert > Class Module and copy Class1 into
'    that module.
' 3. Click Insert > Class Module and copy Class2 into
'    that module.
' 4. Click Insert > Class Module and copy Class3 into
'    that module.
' 5. Open a part, assembly, or drawing.
'
' Postconditions:
' 1. Disables interactively selecting faces in
'    a part or assembly.
'    - or -
'    Disables interactively selecting edges in a
'    drawing.
' 2. Click the Reset button in the Microsoft Visual
'    Basic IDE to re-enable the interactive selection
'    of faces in a part or assembly or edges
'    in a drawing.
'-----
```

'Module

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim partDoc As New Class1
Dim assemblyDoc As New Class2
Dim drawingDoc As New Class3
```

Sub main()

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Determine the document type, then
    ' execute its corresponding class module
    If swModel.GetType = swDocPART Then
        partDoc.init swModel
    ElseIf swModel.GetType = swDocASSEMBLY Then
        assemblyDoc.init swModel
    ElseIf swModel.GetType = swDocDRAWING Then
        drawingDoc.init swModel
    End If
```

End Sub

[Back to top](#)

'Class1

Public WithEvents doc As partDoc

```
Public Function init(ByRef docIn As Object)
    Set doc = docIn
End Function
```

```
Private Function doc_UserSelectionPreNotify(ByVal SelectionType As Long) As Long
    ' Disable the selection of faces in this part
    If SelectionType = swSelFACES Then
        doc_UserSelectionPreNotify = True
    End If
End Function
```

[Back to top](#)

'Class2

Public WithEvents doc As assemblyDoc

```
Public Function init(ByRef docIn As Object)
    Set doc = docIn
End Function
```

```
Private Function doc_UserSelectionPreNotify(ByVal SelectionType As Long) As Long
    ' Disable selection of faces in this assembly
```

```
    If SelectionType = swSelFACES Then  
        doc_UserSelectionPreNotify = True  
    End If  
End Function
```

[Back to top](#)

'Class3

```
Public WithEvents doc As drawingDoc
```

```
Public Function init(ByRef docIn As Object)  
    Set doc = docIn  
End Function
```

```
Private Function doc_UserSelectionPreNotify(ByVal SelectionType As Long) As Long  
    ' Disable the selection of edges in drawings  
    If SelectionType = swSelEDGES Then  
        doc_UserSelectionPreNotify = True  
    End If  
End Function
```

[Back to top](#)

## Disable Selection of Faces and Edges Using a Pre-Notify Event Example (VB.NET)

This example shows how to disable the interactive selection of these entities using a pre-notify event:

- faces in part and assembly documents
- edges in drawing documents

```

'-----
' Preconditions: Open a part, assembly, or drawing.
'
' NOTE: Tools > Options > System > Stop VSTA debugger
' on macro exit must be cleared for this macro
' to run to completion.
'
' Postconditions:
' 1. Disables interactively selecting faces in
'    a part or assembly.
'    - or -
'    Disables interactively selecting edges in a
'    drawing.
' 2. Click the Stop Debugging button in the
'    SOLIDWORKS Visual Studio Tools for
'    Applications IDE to re-enable the
'    interactive selection of faces in
'    a part or assembly document or edges in
'    a drawing document.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Collections
Partial Class SolidWorksMacro
    Public WithEvents pDoc As PartDoc
    Public WithEvents aDoc As AssemblyDoc
    Public WithEvents dDoc As DrawingDoc
    Public Sub main()
        Dim swModel As ModelDoc2
        Dim openPart As Hashtable
        Dim openAssembly As Hashtable
        Dim openDrawing As Hashtable

        swModel = swApp.ActiveDoc

        ' Determine the document type
        ' and set up event handlers
        If swModel.GetType = swDocumentTypes_e.swDocPART Then
            pDoc = swModel
            openPart = New Hashtable
        ElseIf swModel.GetType = swDocumentTypes_e.swDocASSEMBLY Then
            aDoc = swModel
            openAssembly = New Hashtable
        ElseIf swModel.GetType = swDocumentTypes_e.swDocDRAWING Then
            dDoc = swModel
            openDrawing = New Hashtable
        End If
        AttachEventHandlers()
    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        If Not pDoc Is Nothing Then
            AddHandler pDoc.UserSelectionPreNotify, AddressOf Me.pDoc_UserSelectionPreNotify
        End If
        If Not aDoc Is Nothing Then
            AddHandler aDoc.UserSelectionPreNotify, AddressOf Me.aDoc_UserSelectionPreNotify
        End If
        If Not dDoc Is Nothing Then
            AddHandler dDoc.UserSelectionPreNotify, AddressOf Me.dDoc_UserSelectionPreNotify
        End If
    End Sub

    Private Function pDoc_UserSelectionPreNotify(ByVal SelectionType As Integer) As Integer
        ' Disable the selection of faces in this part
        If SelectionType = swSelectType_e.swSelFACES Then
            pDoc_UserSelectionPreNotify = True
        End If
    End Function

    Public Function aDoc_UserSelectionPreNotify(ByVal SelectionType As Integer) As Integer
        ' Disable the selection of faces in this assembly
        If SelectionType = swSelectType_e.swSelFACES Then
            aDoc_UserSelectionPreNotify = True
        End If
    End Function

    Private Function dDoc_UserSelectionPreNotify(ByVal SelectionType As Integer) As Integer
        ' Disable the selection of edges in this drawing
        If SelectionType = swSelectType_e.swSelEDGES Then

```

```
        dDoc_UserSelectionPreNotify = True
    End If
End Function
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class
```



### Disable Wait Cursor When Model View Redrawn (VBA)

This example shows how to suppress the wait cursor (also called the hourglass) when a model view is redrawn.

```
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As Object  
    Dim Part As SldWorks.ModelDoc2  
    Dim modelView As SldWorks.modelView  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set modelView = Part.ActiveView  
    modelView.SuppressWaitCursorDuringRedraw = False  
End Sub
```

## Display Configuration Description in Bill of Materials Example (C#)

This example shows how to traverse an assembly and display the description of each configuration in the bill of materials.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\smartcomponents\pillow_block.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the configurations.
// 2. Gets the name of each configuration and its description.
// 3. Sets and gets whether to display the description of each configuration
//    in the bill of materials.
// 4. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            string[] vConfigNameArr = null;
            Configuration swConfig = default(Configuration);
            Configuration swParentConfig = default(Configuration);
            ConfigurationManager swConfMgr = default(ConfigurationManager);
            object[] vChildConfigArr = null;
            Configuration swChildConfig = default(Configuration);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swConfMgr = (ConfigurationManager)swModel.ConfigurationManager;
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("");
            // Traverse configurations; always at least one configuration exists
            vConfigNameArr = (string[])swModel.GetConfigurationNames();
            foreach (string vConfigName in vConfigNameArr)
            {
                swConfig = (Configuration)swModel.GetConfigurationByName(vConfigName);
                Debug.Print(" Configuration name = " + swConfig.Name);
                Debug.Print(" Description = " + swConfig.Description);
                swConfig.UseDescriptionInBOM = true;
                Debug.Print(" Display description in bill of materials? " + swConfig.UseDescriptionInBOM);
                // Process parent
                swParentConfig = (Configuration)swConfig.GetParent();
                if ((swParentConfig != null))
                {
                    Debug.Print(" Parent = " + swParentConfig.Name);
                }
                // Process children
                vChildConfigArr = (object[])swConfig.GetChildren();
                if ((vChildConfigArr != null))
                {
                    foreach (object vChildConfig in vChildConfigArr)
                    {
                        swChildConfig = (Configuration)vChildConfig;
                        Debug.Print(" Child = " + swChildConfig.Name);
                    }
                }
                Debug.Print("");
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

## Display Configuration Description in Bill of Materials Example (VBA)

This example shows how to traverse an assembly and display the description of each configuration in the bill of materials.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\smartcomponents\pillow_block.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the configurations.
' 2. Gets the name of each configuration and its description.
' 3. Sets and gets whether to display the description of each configuration
'    in the bill of materials.
' 4. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim vConfigNameArr As Variant
    Dim vConfigName As Variant
    Dim swConfig As SldWorks.Configuration
    Dim swParentConfig As SldWorks.Configuration
    Dim swConfMgr As SldWorks.ConfigurationManager
    Dim vChildConfigArr As Variant
    Dim vChildConfig As Variant
    Dim swChildConfig As SldWorks.Configuration

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConfMgr = swModel.ConfigurationManager
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print ""

    ' Traverse configurations; always at least one configuration exists
    vConfigNameArr = swModel.GetConfigurationNames
    For Each vConfigName In vConfigNameArr
        Set swConfig = swModel.GetConfigurationByName(vConfigName)
        Debug.Print "    Configuration name = " & swConfig.Name
        Debug.Print "        Description = " & swConfig.Description
        swConfig.UseDescriptionInBOM = True
        Debug.Print "            Display description in bill of materials? " & swConfig.UseDescriptionInBOM

        ' Process parent
        Set swParentConfig = swConfig.GetParent
        If Not swParentConfig Is Nothing Then
            Debug.Print "                Parent = " & swParentConfig.Name
        End If

        ' Process children
        vChildConfigArr = swConfig.GetChildren
        If Not IsEmpty(vChildConfigArr) Then
            For Each vChildConfig In vChildConfigArr
                Set swChildConfig = vChildConfig
                Debug.Print "                    Child = " & swChildConfig.Name
            Next
        End If
        Debug.Print ""
    Next

End Sub

```

## Display Configuration Description in Bill of Materials Example (VB.NET)

This example shows how to traverse an assembly and display the description of each configuration in the bill of materials.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\smartcomponents\pillow_block.sldasm.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Traverses the configurations.
' 2. Gets the name of each configuration and its description.
' 3. Sets and gets whether to display the description of each configuration
'    in the bill of materials.
' 4. Examine the Immediate window.
'-----
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim vConfigNameArr() As Object
        Dim vConfigName As Object
        Dim swConfig As Configuration
        Dim swParentConfig As Configuration
        Dim swConfMgr As ConfigurationManager
        Dim vChildConfigArr() As Object
        Dim vChildConfig As Object
        Dim swChildConfig As Configuration

        swModel = swApp.ActiveDoc
        swConfMgr = swModel.ConfigurationManager
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("")
        ' Traverse configurations; always at least one configuration exists
        vConfigNameArr = swModel.GetConfigurationNames
        For Each vConfigName In vConfigNameArr
            swConfig = swModel.GetConfigurationByName(vConfigName)
            Debug.Print(" Configuration name = " & swConfig.Name)
            Debug.Print(" Description = " & swConfig.Description)
            swConfig.UseDescriptionInBOM = True
            Debug.Print(" Display description in bill of materials? " & swConfig.UseDescriptionInBOM)
            ' Process parent
            swParentConfig = swConfig.GetParent
            If Not swParentConfig Is Nothing Then
                Debug.Print(" Parent = " & swParentConfig.Name)
            End If
            ' Process children
            vChildConfigArr = swConfig.GetChildren
            If Not IsNothing(vChildConfigArr) Then
                For Each vChildConfig In vChildConfigArr
                    swChildConfig = vChildConfig
                    Debug.Print(" Child = " & swChildConfig.Name)
                Next
            End If
            Debug.Print("")
        Next

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```